Object Oriented Analysis and Design

Mini Project

Museum Management System

https://github.com/Nehal-bit/museum_management_system

Name	SRN
Nehal G	PES2UG23CS380
Nevin Mathew Thomas	PES2UG23CS381
Pinisetti Sudhiksha	PES2UG23CS916

Description

The Museum Management System is a centralized software solution designed to automate and streamline the management of museum operations such as exhibit handling, visitor scheduling, guide assignment, ticketing, and analytics. The system supports multiple user roles including Admin, Guide, and Visitor, each with clearly defined responsibilities and access privileges.

MVC Architecture

The Museum Management System is built using Spring Boot and follows the MVC (Model-View-Controller) architecture. Each layer has a clear role, and responsibilities are well separated.

Model:

- Represents application data and business logic
- Includes :
  - **Entities** : Booking, Ticket, User, Visitor, Guide, Exhibit, Schedule
  - **Repositories** : Handle database operations (BookingRepository, TicketRepository, etc.)
  - **Services** : Contain business logic (BookingService, UserService, TicketPricingService, NotificationManager)

View:

- Implemented as JSON responses in a REST API
- Responsible for sending structured data to the client

Controller:

- Handles HTTP requests and responses
- Includes : BookingController, AdminController, AuthController
- Connects View and Model via Service layer

GRASP PRINCIPLES

1. Information Expert:

- Responsibility assigned to the class with required data
  - BookingService handles booking validation and capacity checks
  - TicketPricingService manages ticket pricing logic

2. Controller:

● A single class manages system-level operations
 - BookingService handles the complete booking workflow (create, confirm, cancel, etc.)

3. High Cohesion:

- BookingService handles only booking-related operations (create, confirm, cancel, assign)
- UserService handles only user-related operations (register, login, manage users)

DESIGN PATTERNS

1. Singleton:

● Ensures only one instance of a class exists

- NotificationManager uses a `static instance` and `getInstance()` method

2. Factory (User Creation):

● Creates objects based on conditions:

- UserService.register() creates Visitor or Guide depending on role

3. Factory (Booking Creation):

● Encapsulates complex object creation

- BookingService.createBooking() creates Schedule, Booking, and Ticket together

CODE

1. Application Entry Point

1.1 Main Class

- MuseumAppApplication.java

```
package com.museum;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class MuseumAppApplication {

    public static void main(String[] args) {

        SpringApplication.run(MuseumAppApplication.class, args);

    }

}
```

1.2 Config

- DataSeeder.java

```
package com.museum.config;

import com.museum.model.Admin;

import com.museum.model.UserRole;

import com.museum.repository.AdminRepository;

import org.springframework.boot.CommandLineRunner;

import org.springframework.stereotype.Component;

/**

 * Seeds a default Admin account on startup.

 * Default: admin@museum.com / admin123

 */

@Component
```

```java
public class DataSeeder implements CommandLineRunner {



    private final AdminRepository adminRepository;



    public DataSeeder(AdminRepository adminRepository) {

        this.adminRepository = adminRepository;

    }



    @Override

    public void run(String... args) {

        if (adminRepository.findByEmail("admin@museum.com").isEmpty()) {

            Admin admin = new Admin();

            admin.setName("System Admin");

            admin.setEmail("admin@museum.com");

            admin.setPassword("admin123");

            admin.setRole(UserRole.ADMIN);

            adminRepository.save(admin);

            System.out.println("Default admin created: admin@museum.com / admin123");

        }

    }

}
```

- GlobalExceptionHandler.java

```java
package com.museum.config;



import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.ExceptionHandler;

import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.LocalDateTime;

import java.util.Map;
```

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<Map<String, Object>> handleRuntime(RuntimeException ex) {
        return ResponseEntity.badRequest().body(Map.of(
                "timestamp", LocalDateTime.now().toString(),
                "error", ex.getMessage() != null ? ex.getMessage() : "Request failed"
        ));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleGeneric(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(Map.of(
                "timestamp", LocalDateTime.now().toString(),
                "error", "Internal server error"
        ));
    }
}
```

2. User Module

2.1 Model

- User.java

```java
package com.museum.model;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;

@Entity
@Table(name = "users")
```

```java
@Inheritance(strategy = InheritanceType.JOINED)

public abstract class User {



    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long userId;



    @Column(nullable = false)

    private String name;



    @Column(nullable = false, unique = true)

    private String email;



    @JsonIgnore

    @Column(nullable = false)

    private String password;



    @Enumerated(EnumType.STRING)

    @Column(nullable = false)

    private UserRole role;



    public Long getUserId() { return userId; }

    public void setUserId(Long userId) { this.userId = userId; }



    public String getName() { return name; }

    public void setName(String name) { this.name = name; }



    public String getEmail() { return email; }

    public void setEmail(String email) { this.email = email; }



    public String getPassword() { return password; }
```

```java
    public void setPassword(String password) { this.password = password; }



    public UserRole getRole() { return role; }

    public void setRole(UserRole role) { this.role = role; }

}
```

- UserRole.java

```java
package com.museum.model;

public enum UserRole {

    ADMIN,

    VISITOR,

    GUIDE

}
```

- Admin.java

```java
package com.museum.model;



import jakarta.persistence.*;



@Entity

@Table(name = "admins")

@PrimaryKeyJoinColumn(name = "user_id")

public class Admin extends User {

    @PrePersist

    protected void prePersist() {

        this.setRole(UserRole.ADMIN);

    }

}
```

- Visitor.java

```java
package com.museum.model;
```

```java
import com.fasterxml.jackson.annotation.JsonIgnore;

import jakarta.persistence.*;

import java.time.LocalDateTime;

import java.util.ArrayList;

import java.util.List;



@Entity

@Table(name = "visitors")

@PrimaryKeyJoinColumn(name = "user_id")

public class Visitor extends User {



    private String phone;

    private LocalDateTime registeredOn;



    @JsonIgnore

    @OneToMany(mappedBy = "visitor", cascade = CascadeType.ALL, fetch = FetchType.LAZY)

    private List<Booking> bookings = new ArrayList<>();



    @JsonIgnore

    @OneToMany(mappedBy = "visitor", cascade = CascadeType.ALL, fetch = FetchType.LAZY)

    private List<Feedback> feedbacks = new ArrayList<>();



    @PrePersist

    protected void prePersist() {

        this.registeredOn = LocalDateTime.now();

        this.setRole(UserRole.VISITOR);

    }



    public String getPhone() { return phone; }

    public void setPhone(String phone) { this.phone = phone; }

    public LocalDateTime getRegisteredOn() { return registeredOn; }
```

```java
    public void setRegisteredOn(LocalDateTime registeredOn) { this.registeredOn = registeredOn; }

    public List<Booking> getBookings() { return bookings; }

    public void setBookings(List<Booking> bookings) { this.bookings = bookings; }

    public List<Feedback> getFeedbacks() { return feedbacks; }

    public void setFeedbacks(List<Feedback> feedbacks) { this.feedbacks = feedbacks; }

}
```

- Guide.java

```java
package com.museum.model;


import com.fasterxml.jackson.annotation.JsonIgnore;

import jakarta.persistence.*;

import java.util.ArrayList;

import java.util.List;


@Entity

@Table(name = "guides")

@PrimaryKeyJoinColumn(name = "user_id")

public class Guide extends User {


    private String specialization;


    @Column(nullable = false)

    private boolean available = true;


    @JsonIgnore

    @OneToMany(mappedBy = "guide", cascade = CascadeType.ALL, fetch = FetchType.LAZY)

    private List<GuideAssignment> assignments = new ArrayList<>();


    @PrePersist
```

```
    protected void prePersist() {

        this.setRole(UserRole.GUIDE);

    }



    public String getSpecialization() { return specialization; }

    public void setSpecialization(String specialization) { this.specialization = specialization; }

    public boolean isAvailable() { return available; }

    public void setAvailable(boolean available) { this.available = available; }

    public List<GuideAssignment> getAssignments() { return assignments; }

    public void setAssignments(List<GuideAssignment> assignments) { this.assignments = assignments; }

}
```

2.2 Controller

- AuthController.java

```
package com.museum.controller;



import com.museum.dto.LoginDTO;

import com.museum.dto.RegisterDTO;

import com.museum.model.User;

import com.museum.model.UserRole;

import com.museum.service.UserService;

import jakarta.validation.Valid;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;



import java.util.Map;



/**

 * Public registration: VISITOR only.
```

```
 * Guide registration: Admin-only via /api/admin/guides/create.

 */

@RestController

@RequestMapping("/api/auth")

public class AuthController {



    private final UserService userService;



    public AuthController(UserService userService) {

        this.userService = userService;

    }



    @PostMapping("/register")

    public ResponseEntity<?> register(@Valid @RequestBody RegisterDTO dto) {

        // Block guide self-registration — guides are created by admin only

        if (dto.getRole() == UserRole.GUIDE || dto.getRole() == UserRole.ADMIN) {

            return ResponseEntity.status(HttpStatus.FORBIDDEN)

                    .body(Map.of("error", "Public registration is only available for Visitors. Guides are
registered by Admin.")); 

        }

        try {

            User user = userService.register(dto);

            return ResponseEntity.status(HttpStatus.CREATED).body(Map.of(

                    "message", "Registration successful",

                    "userId", user.getUserId(),

                    "id",     user.getUserId(),

                    "name",   user.getName(),

                    "email",  user.getEmail(),

                    "role",   user.getRole()

            ));

        } catch (RuntimeException e) {
```

```
            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));

        }

    }



    @PostMapping("/login")

    public ResponseEntity<?> login(@Valid @RequestBody LoginDTO dto) {

        try {

            User user = userService.login(dto);

            return ResponseEntity.ok(Map.of(

                    "message", "Login successful",

                    "userId",  user.getUserId(),

                    "id",      user.getUserId(),

                    "name",    user.getName(),

                    "email",   user.getEmail(),

                    "role",    user.getRole()

            ));

        } catch (RuntimeException e) {

            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)

                    .body(Map.of("error", e.getMessage()));

        }

    }

}
```

- VisitorController.java

```
package com.museum.controller;



import com.museum.model.Exhibit;

import com.museum.model.Notification;

import com.museum.service.*;

import org.springframework.http.ResponseEntity;
```

```java
import org.springframework.web.bind.annotation.*;



import java.util.List;

import java.util.Map;



@RestController

@RequestMapping("/api/visitors")

public class VisitorController {



    private final ExhibitService exhibitService;

    private final BookingService bookingService;

    private final FeedbackService feedbackService;

    private final NotificationManager notificationManager;



    public VisitorController(ExhibitService exhibitService, BookingService bookingService,

                             FeedbackService feedbackService, NotificationManager notificationManager) {

        this.exhibitService = exhibitService;

        this.bookingService = bookingService;

        this.feedbackService = feedbackService;

        this.notificationManager = notificationManager;

    }



    @GetMapping("/{id}/exhibits")

    public ResponseEntity<List<Exhibit>> browseExhibits(@PathVariable Long id) {

        return ResponseEntity.ok(exhibitService.getActiveExhibits());

    }



    @GetMapping("/{id}/bookings")

    public ResponseEntity<?> getMyBookings(@PathVariable Long id) {

        return ResponseEntity.ok(bookingService.getBookingsByVisitorFlat(id));

    }
```

```
    @GetMapping("/{id}/notifications")
    public ResponseEntity<List<Notification>> getMyNotifications(@PathVariable Long id) {
        return ResponseEntity.ok(notificationManager.getNotificationsForUser(id));
    }

    @GetMapping("/{id}/unread-count")
    public ResponseEntity<Map<String, Object>> getUnreadCount(@PathVariable Long id) {
        return ResponseEntity.ok(Map.of("visitorId", id,
                "unreadNotifications", notificationManager.countUnread(id)));
    }
}
```

2.3 Service

- UserService.java

```
package com.museum.service;

import com.museum.dto.LoginDTO;
import com.museum.dto.RegisterDTO;
import com.museum.model.*;
import com.museum.repository.*;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;

@Service
public class UserService {

    private final UserRepository userRepository;
```

```java
    private final VisitorRepository visitorRepository;

    private final GuideRepository guideRepository;

    private final AdminRepository adminRepository;



    public UserService(UserRepository userRepository,

                       VisitorRepository visitorRepository,

                       GuideRepository guideRepository,

                       AdminRepository adminRepository) {

        this.userRepository = userRepository;

        this.visitorRepository = visitorRepository;

        this.guideRepository = guideRepository;

        this.adminRepository = adminRepository;

    }



    @Transactional

    public User register(RegisterDTO dto) {

        if (userRepository.existsByEmail(dto.getEmail())) {

            throw new RuntimeException("Email already registered: " + dto.getEmail());

        }

        if (dto.getRole() == UserRole.VISITOR) {

            Visitor v = new Visitor();

            v.setName(dto.getName());

            v.setEmail(dto.getEmail());

            v.setPassword(dto.getPassword());

            v.setPhone(dto.getPhone());

            return visitorRepository.save(v);

        } else if (dto.getRole() == UserRole.GUIDE) {

            Guide g = new Guide();

            g.setName(dto.getName());

            g.setEmail(dto.getEmail());

            g.setPassword(dto.getPassword());
```

```
        g.setSpecialization(dto.getSpecialization());
        return guideRepository.save(g);
    } else {
        throw new RuntimeException("Invalid role for registration.");
    }
}

@Transactional(readOnly = true)
public User login(LoginDTO dto) {
    return userRepository.findByEmailAndPassword(dto.getEmail(), dto.getPassword())
            .orElseThrow(() -> new RuntimeException("Invalid email or password."));
}

@Transactional(readOnly = true) public List<User> getAllUsers() { return userRepository.findAll(); }

@Transactional
public void deleteGuide(Long guideId) {
    Guide guide = guideRepository.findById(guideId)
            .orElseThrow(() -> new RuntimeException("Guide not found: " + guideId));
    // Check for active (PENDING or CONFIRMED) assignments
    boolean hasActive = guide.getAssignments().stream()
            .anyMatch(a -> "PENDING".equals(a.getStatus()) || "CONFIRMED".equals(a.getStatus()));
    if (hasActive) {
        throw new RuntimeException(
            "Cannot delete guide with active assignments. " +
            "Reassign or cancel those bookings first.");
    }
    guideRepository.delete(guide);
}

@Transactional(readOnly = true)
```

```
    public List<User> getUsersByRole(UserRole role) { return userRepository.findByRole(role); }



    @Transactional(readOnly = true)

    public User getUserById(Long id) {

        return userRepository.findById(id)

                .orElseThrow(() -> new RuntimeException("User not found: " + id));

    }

}
```

2.4 Repository

- UserRepository.java

```
package com.museum.repository;



import com.museum.model.User;

import com.museum.model.UserRole;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;



import java.util.List;

import java.util.Optional;



@Repository

public interface UserRepository extends JpaRepository<User, Long> {



    Optional<User> findByEmail(String email);



    Optional<User> findByEmailAndPassword(String email, String password);



    boolean existsByEmail(String email);
```

```
    List<User> findByRole(UserRole role);

}
```

- VisitorRepository.java

```
package com.museum.repository;


import com.museum.model.Visitor;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;


import java.util.Optional;


@Repository

public interface VisitorRepository extends JpaRepository<Visitor, Long> {


    Optional<Visitor> findByEmail(String email);


    boolean existsByEmail(String email);

}
```

- AdminRepository.java

```
package com.museum.repository;


import com.museum.model.Admin;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;


import java.util.Optional;
```

```
@Repository
public interface AdminRepository extends JpaRepository<Admin, Long> {

    Optional<Admin> findByEmail(String email);

    Optional<Admin> findByEmailAndPassword(String email, String password);
}
```

2.5 DTO

- RegisterDTO.java

```
package com.museum.dto;

import com.museum.model.UserRole;

public class RegisterDTO {
    private String name;
    private String email;
    private String password;
    private UserRole role;
    private String phone;
    private String specialization;

    public String getName() { return name; }
    public void setName(String v) { this.name = v; }
    public String getEmail() { return email; }
    public void setEmail(String v) { this.email = v; }
    public String getPassword() { return password; }
    public void setPassword(String v) { this.password = v; }
```

```
    public UserRole getRole() { return role; }

    public void setRole(UserRole v) { this.role = v; }

    public String getPhone() { return phone; }

    public void setPhone(String v) { this.phone = v; }

    public String getSpecialization() { return specialization; }

    public void setSpecialization(String v) { this.specialization = v; }

}
```

- LoginDTO.java

```
package com.museum.dto;



public class LoginDTO {

    private String email;

    private String password;


    public String getEmail() { return email; }

    public void setEmail(String v) { this.email = v; }

    public String getPassword() { return password; }

    public void setPassword(String v) { this.password = v; }

}
```

3. Exhibit Module

3.1 Model

- Exhibit.java

```
package com.museum.model;



import com.fasterxml.jackson.annotation.JsonIgnore;
```

```java
import jakarta.persistence.*;

import java.time.LocalDate;

import java.util.ArrayList;

import java.util.List;



@Entity

@Table(name = "exhibits")

public class Exhibit {



    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long exhibitId;



    @Column(nullable = false)

    private String name;



    @Column(columnDefinition = "TEXT")

    private String description;



    private String category;

    private LocalDate startDate;

    private LocalDate endDate;



    @Column(nullable = false)

    private String status = "ACTIVE";



    @Column(nullable = false)

    private int maxOccupancy = 50;



    @JsonIgnore

    @OneToMany(mappedBy = "exhibit", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
```

```java
    private List<Schedule> schedules = new ArrayList<>();



    @JsonIgnore

    @OneToMany(mappedBy = "exhibit", cascade = CascadeType.ALL, fetch = FetchType.LAZY)

    private List<Feedback> feedbacks = new ArrayList<>();



    public Long getExhibitId() { return exhibitId; }

    public void setExhibitId(Long exhibitId) { this.exhibitId = exhibitId; }

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }

    public String getDescription() { return description; }

    public void setDescription(String description) { this.description = description; }

    public String getCategory() { return category; }

    public void setCategory(String category) { this.category = category; }

    public LocalDate getStartDate() { return startDate; }

    public void setStartDate(LocalDate startDate) { this.startDate = startDate; }

    public LocalDate getEndDate() { return endDate; }

    public void setEndDate(LocalDate endDate) { this.endDate = endDate; }

    public String getStatus() { return status; }

    public void setStatus(String status) { this.status = status; }

    public int getMaxOccupancy() { return maxOccupancy; }

    public void setMaxOccupancy(int maxOccupancy) { this.maxOccupancy = maxOccupancy; }

    public List<Schedule> getSchedules() { return schedules; }

    public void setSchedules(List<Schedule> schedules) { this.schedules = schedules; }

    public List<Feedback> getFeedbacks() { return feedbacks; }

    public void setFeedbacks(List<Feedback> feedbacks) { this.feedbacks = feedbacks; }

}
```

3.2 Controller

- ExhibitController.java

```java
package com.museum.controller;


import com.museum.model.Exhibit;

import com.museum.service.ExhibitService;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;


import java.util.List;

import java.util.Map;


@RestController

@RequestMapping("/api/exhibits")

public class ExhibitController {


    private final ExhibitService exhibitService;


    public ExhibitController(ExhibitService exhibitService) {

        this.exhibitService = exhibitService;

    }


    @GetMapping({"", "/"})

    public ResponseEntity<List<Exhibit>> getAllExhibits() {

        return ResponseEntity.ok(exhibitService.getAllExhibits());

    }


    @GetMapping("/active")

    public ResponseEntity<List<Exhibit>> getActiveExhibits() {

        return ResponseEntity.ok(exhibitService.getActiveExhibits());

    }
```

```
    @GetMapping("/{id}")

    public ResponseEntity<?> getExhibitById(@PathVariable Long id) {

        try { return ResponseEntity.ok(exhibitService.getExhibitById(id)); }

        catch (RuntimeException e) { return ResponseEntity.notFound().build(); }

    }



    @GetMapping("/search")

    public ResponseEntity<List<Exhibit>> searchExhibits(@RequestParam String name) {

        return ResponseEntity.ok(exhibitService.searchByName(name));

    }



    @GetMapping("/category")

    public ResponseEntity<List<Exhibit>> getByCategory(@RequestParam String category) {

        return ResponseEntity.ok(exhibitService.getByCategory(category));

    }



    @PostMapping({"", "/"})

    public ResponseEntity<?> addExhibit(@RequestBody Exhibit exhibit) {

        try {

            Exhibit saved = exhibitService.addExhibit(exhibit);

            return ResponseEntity.status(HttpStatus.CREATED).body(saved);

        } catch (RuntimeException e) {

            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));

        }

    }



    @PutMapping("/{id}")

    public ResponseEntity<?> updateExhibit(@PathVariable Long id, @RequestBody Exhibit exhibit) {

        try { return ResponseEntity.ok(exhibitService.updateExhibit(id, exhibit)); }

        catch (RuntimeException e) { return ResponseEntity.badRequest().body(Map.of("error",
e.getMessage())); }
```

```
    }



    @DeleteMapping("/{id}")

    public ResponseEntity<?> deleteExhibit(@PathVariable Long id) {

        try { exhibitService.deleteExhibit(id); return ResponseEntity.ok(Map.of("message", "Exhibit
archived.")); }

        catch (RuntimeException e) { return ResponseEntity.badRequest().body(Map.of("error",
e.getMessage())); }

    }

}
```

3.3 Service

- ExhibitService.java

```
package com.museum.service;



import com.museum.model.Exhibit;

import com.museum.repository.ExhibitRepository;

import org.springframework.stereotype.Service;

import org.springframework.transaction.annotation.Transactional;



import java.util.List;



@Service

public class ExhibitService {



    private final ExhibitRepository exhibitRepository;



    public ExhibitService(ExhibitRepository exhibitRepository) {

        this.exhibitRepository = exhibitRepository;

    }



    @Transactional
```

```java
public Exhibit addExhibit(Exhibit exhibit) {

    if (exhibit.getStatus() == null || exhibit.getStatus().isBlank()) {

        exhibit.setStatus("ACTIVE");

    }

    if (exhibit.getMaxOccupancy() <= 0) {

        exhibit.setMaxOccupancy(50);

    }

    return exhibitRepository.save(exhibit);

}



@Transactional

public Exhibit updateExhibit(Long id, Exhibit updated) {

    Exhibit existing = getExhibitById(id);

    existing.setName(updated.getName());

    existing.setDescription(updated.getDescription());

    existing.setCategory(updated.getCategory());

    existing.setStartDate(updated.getStartDate());

    existing.setEndDate(updated.getEndDate());

    if (updated.getStatus() != null) existing.setStatus(updated.getStatus());

    if (updated.getMaxOccupancy() > 0) existing.setMaxOccupancy(updated.getMaxOccupancy());

    return exhibitRepository.save(existing);

}



@Transactional

public void deleteExhibit(Long id) {

    Exhibit exhibit = getExhibitById(id);

    exhibit.setStatus("ARCHIVED");

    exhibitRepository.save(exhibit);

}



@Transactional
```

```java
    public void hardDeleteExhibit(Long id) {

        exhibitRepository.deleteById(id);

    }



    @Transactional(readOnly = true)

    public List<Exhibit> getAllExhibits() {

        return exhibitRepository.findAll();

    }



    @Transactional(readOnly = true)

    public List<Exhibit> getActiveExhibits() {

        return exhibitRepository.findByStatus("ACTIVE");

    }



    @Transactional(readOnly = true)

    public Exhibit getExhibitById(Long id) {

        return exhibitRepository.findById(id)

                .orElseThrow(() -> new RuntimeException("Exhibit not found: " + id));

    }



    @Transactional(readOnly = true)

    public List<Exhibit> searchByName(String name) {

        return exhibitRepository.findByNameContainingIgnoreCase(name);

    }



    @Transactional(readOnly = true)

    public List<Exhibit> getByCategory(String category) {

        return exhibitRepository.findByCategory(category);

    }

}
```

3.4 Repository

- ExhibitRepository.java

```java
package com.museum.repository;


import com.museum.model.Exhibit;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;


import java.util.List;


@Repository

public interface ExhibitRepository extends JpaRepository<Exhibit, Long> {


    List<Exhibit> findByStatus(String status);


    List<Exhibit> findByCategory(String category);


    List<Exhibit> findByNameContainingIgnoreCase(String name);

}
```

4. Booking Module

4.1 Model

- Booking.java

```java
package com.museum.model;


import com.fasterxml.jackson.annotation.JsonIgnore;

import jakarta.persistence.*;

import java.time.LocalDateTime;

import java.util.ArrayList;

import java.util.List;
```

```java
@Entity
@Table(name = "bookings")
public class Booking {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long bookingId;

    @Column(nullable = false, updatable = false)
    private LocalDateTime bookingDate;

    @Column(nullable = false)
    private String status = "PENDING";

    @Column(nullable = false)
    private int numberOfTickets = 1;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "visitor_id", nullable = false)
    private Visitor visitor;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "schedule_id", nullable = false)
    private Schedule schedule;

    @JsonIgnore
    @OneToMany(mappedBy = "booking", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<Ticket> tickets = new ArrayList<>();

    @JsonIgnore
```

```java
    @OneToMany(mappedBy = "booking", cascade = CascadeType.ALL, fetch = FetchType.LAZY)

    private List<GuideAssignment> guideAssignments = new ArrayList<>();



    @PrePersist

    protected void prePersist() { this.bookingDate = LocalDateTime.now(); }



    public Long getBookingId() { return bookingId; }

    public void setBookingId(Long bookingId) { this.bookingId = bookingId; }

    public LocalDateTime getBookingDate() { return bookingDate; }

    public void setBookingDate(LocalDateTime bookingDate) { this.bookingDate = bookingDate; }

    public String getStatus() { return status; }

    public void setStatus(String status) { this.status = status; }

    public int getNumberOfTickets() { return numberOfTickets; }

    public void setNumberOfTickets(int numberOfTickets) { this.numberOfTickets = numberOfTickets; }

    public Visitor getVisitor() { return visitor; }

    public void setVisitor(Visitor visitor) { this.visitor = visitor; }

    public Schedule getSchedule() { return schedule; }

    public void setSchedule(Schedule schedule) { this.schedule = schedule; }

    public List<Ticket> getTickets() { return tickets; }

    public void setTickets(List<Ticket> tickets) { this.tickets = tickets; }

    public List<GuideAssignment> getGuideAssignments() { return guideAssignments; }

    public void setGuideAssignments(List<GuideAssignment> guideAssignments) { this.guideAssignments =
guideAssignments; }

}
```

- Schedule.java

```java
package com.museum.model;



import com.fasterxml.jackson.annotation.JsonIgnore;

import jakarta.persistence.*;

import java.time.LocalDate;
```

```java
import java.time.LocalTime;

import java.util.ArrayList;

import java.util.List;



@Entity

@Table(name = "schedules")

public class Schedule {



    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long scheduleId;



    @Column(nullable = false)

    private LocalDate visitDate;



    private LocalTime startTime;

    private LocalTime endTime;



    @Column(nullable = false)

    private int maxVisitors = 30;



    @Column(nullable = false)

    private int currentVisitors = 0;



    @Enumerated(EnumType.STRING)

    @Column(nullable = false)

    private ScheduleStatus status = ScheduleStatus.PENDING;



    @ManyToOne(fetch = FetchType.LAZY)

    @JoinColumn(name = "exhibit_id", nullable = false)

    private Exhibit exhibit;
```

```java
@ManyToOne(fetch = FetchType.LAZY)

@JoinColumn(name = "requested_by")

private Visitor requestedBy;



@JsonIgnore

@OneToMany(mappedBy = "schedule", cascade = CascadeType.ALL, fetch = FetchType.LAZY)

private List<Booking> bookings = new ArrayList<>();



public boolean hasAvailableSlots() { return currentVisitors < maxVisitors; }

public int availableSlots() { return maxVisitors - currentVisitors; }



public Long getScheduleId() { return scheduleId; }

public void setScheduleId(Long scheduleId) { this.scheduleId = scheduleId; }

public LocalDate getVisitDate() { return visitDate; }

public void setVisitDate(LocalDate visitDate) { this.visitDate = visitDate; }

public LocalTime getStartTime() { return startTime; }

public void setStartTime(LocalTime startTime) { this.startTime = startTime; }

public LocalTime getEndTime() { return endTime; }

public void setEndTime(LocalTime endTime) { this.endTime = endTime; }

public int getMaxVisitors() { return maxVisitors; }

public void setMaxVisitors(int maxVisitors) { this.maxVisitors = maxVisitors; }

public int getCurrentVisitors() { return currentVisitors; }

public void setCurrentVisitors(int currentVisitors) { this.currentVisitors = currentVisitors; }

public ScheduleStatus getStatus() { return status; }

public void setStatus(ScheduleStatus status) { this.status = status; }

public Exhibit getExhibit() { return exhibit; }

public void setExhibit(Exhibit exhibit) { this.exhibit = exhibit; }

public Visitor getRequestedBy() { return requestedBy; }

public void setRequestedBy(Visitor requestedBy) { this.requestedBy = requestedBy; }

public List<Booking> getBookings() { return bookings; }
```

```
    public void setBookings(List<Booking> bookings) { this.bookings = bookings; }

}
```

- ScheduleStatus.java

```
package com.museum.model;


public enum ScheduleStatus {

    PENDING,

    APPROVED,

    REJECTED

}
```

4.2 Controller

- BookingController.java

```
package com.museum.controller;


import com.museum.model.TicketType;

import com.museum.service.BookingService;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;


import java.time.LocalDate;

import java.time.LocalTime;

import java.util.Map;


@RestController

@RequestMapping("/api/bookings")

public class BookingController {
```

```java
    private final BookingService bookingService;



    public BookingController(BookingService bookingService) {

        this.bookingService = bookingService;

    }



    @PostMapping({"", "/"})

    public ResponseEntity<?> createBooking(@RequestBody Map<String, Object> body) {

        try {

            Long visitorId = Long.valueOf(body.get("visitorId").toString());

            Long exhibitId = Long.valueOf(body.get("exhibitId").toString());

            String dateStr = body.get("visitDate").toString();

            String timeStr = body.getOrDefault("visitTime", "10:00").toString();



            LocalDate visitDate = LocalDate.parse(dateStr);

            LocalTime visitTime;

            try { visitTime = LocalTime.parse(timeStr); }

            catch (Exception e) { visitTime = LocalTime.of(10, 0); }



            if (body.containsKey("ticketBreakdown") && body.get("ticketBreakdown") instanceof Map) {

                @SuppressWarnings("unchecked")

                Map<String, Object> ticketBreakdown = (Map<String, Object>) body.get("ticketBreakdown");

                Map<String, Object> result = bookingService.createBookingWithMultipleTypes(

                        visitorId, exhibitId, visitDate, visitTime, ticketBreakdown);

                return ResponseEntity.status(HttpStatus.CREATED).body(result);

            } else {

                String typeStr = body.getOrDefault("ticketType", "ADULT").toString();

                int numberOfTickets;

                try {

                    numberOfTickets = Integer.parseInt(body.getOrDefault("numberOfTickets",
"1").toString());
```

```
                } catch (NumberFormatException e) {

                    return ResponseEntity.badRequest().body(Map.of("error", "numberOfTickets must be a
valid integer."));

                }

                if (numberOfTickets < 1)

                    return ResponseEntity.badRequest().body(Map.of("error", "Number of tickets must be at
least 1."));



                TicketType ticketType;

                try { ticketType = TicketType.valueOf(typeStr.toUpperCase()); }

                catch (Exception e) { ticketType = TicketType.ADULT; }



                Map<String, Object> result = bookingService.createBooking(

                        visitorId, exhibitId, visitDate, visitTime, ticketType, numberOfTickets);

                return ResponseEntity.status(HttpStatus.CREATED).body(result);

            }



        } catch (IllegalArgumentException e) {

            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));

        } catch (RuntimeException e) {

            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));

        }

    }



    @GetMapping({"", "/"})

    public ResponseEntity<?> getAllBookings() {

        return ResponseEntity.ok(bookingService.getAllBookingsFlat());

    }



    @GetMapping("/{id}")

    public ResponseEntity<?> getById(@PathVariable Long id) {

        try { return ResponseEntity.ok(bookingService.getBookingFlat(id)); }
```

```
        catch (RuntimeException e) { return ResponseEntity.notFound().build(); }

    }



    @GetMapping("/visitor/{visitorId}")

    public ResponseEntity<?> getByVisitor(@PathVariable Long visitorId) {

        return ResponseEntity.ok(bookingService.getBookingsByVisitorFlat(visitorId));

    }



    @PutMapping("/{id}/cancel")

    public ResponseEntity<?> cancel(@PathVariable Long id) {

        try { return ResponseEntity.ok(bookingService.cancelBooking(id)); }

        catch (RuntimeException e) { return ResponseEntity.badRequest().body(Map.of("error",
e.getMessage())); }

    }

}
```

- ScheduleController.java

```
package com.museum.controller;



import com.museum.service.BookingService;

import com.museum.service.ScheduleService;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;



import java.util.Map;



@RestController

@RequestMapping("/api/schedules")

public class ScheduleController {



    private final ScheduleService scheduleService;
```

```java
    private final BookingService bookingService;



    public ScheduleController(ScheduleService scheduleService, BookingService bookingService) {

        this.scheduleService = scheduleService;

        this.bookingService = bookingService;

    }



    @GetMapping("/visitor/{visitorId}")

    public ResponseEntity<?> getByVisitor(@PathVariable Long visitorId) {

        try {

            return ResponseEntity.ok(bookingService.getBookingsByVisitorFlat(visitorId));

        } catch (RuntimeException e) {

            return ResponseEntity.ok(java.util.List.of());

        }

    }



    @GetMapping

    public ResponseEntity<?> getAllSchedules() {

        return ResponseEntity.ok(bookingService.getAllBookingsFlat());

    }



    @GetMapping("/pending")

    public ResponseEntity<?> getPending() {

        return ResponseEntity.ok(bookingService.getPendingBookingsFlat());

    }



    @PutMapping("/{id}/approve")

    public ResponseEntity<?> approve(@PathVariable Long id) {

        try { return ResponseEntity.ok(bookingService.confirmBooking(id)); }

        catch (RuntimeException e) { return ResponseEntity.badRequest().body(Map.of("error",
e.getMessage())); }
```

```
    }



    @PutMapping("/{id}/reject")

    public ResponseEntity<?> reject(@PathVariable Long id) {

        try { return ResponseEntity.ok(bookingService.rejectBooking(id)); }

        catch (RuntimeException e) { return ResponseEntity.badRequest().body(Map.of("error",
e.getMessage())); }

    }



    @PostMapping("/request")

    public ResponseEntity<?> legacyRequest(@RequestBody Map<String, Object> body) {

        try {

            Long visitorId  = Long.valueOf(body.get("visitorId").toString());

            Long exhibitId  = Long.valueOf(body.get("exhibitId").toString());

            String dateStr  = body.getOrDefault("visitDate", "").toString();

            String timeStr  = body.getOrDefault("visitTime",

                              body.getOrDefault("startTime", "10:00")).toString();

            int numberOfTickets = body.get("numberOfTickets") != null ?

                    Integer.parseInt(body.get("numberOfTickets").toString()) : 1;



            java.time.LocalDate visitDate;

            try { visitDate = java.time.LocalDate.parse(dateStr); }

            catch (Exception e) { return ResponseEntity.badRequest().body(Map.of("error", "Invalid
visitDate")); }



            java.time.LocalTime visitTime;

            try { visitTime = java.time.LocalTime.parse(timeStr); }

            catch (Exception e) { visitTime = java.time.LocalTime.of(10, 0); }



            Map<String, Object> result = bookingService.createBooking(

                    visitorId, exhibitId, visitDate, visitTime,

                    com.museum.model.TicketType.ADULT, numberOfTickets);
```

```
            return org.springframework.http.ResponseEntity.status(201).body(result);

        } catch (RuntimeException e) {

            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));

        }

    }

}
```

4.3 Service

- BookingService.java

```
package com.museum.service;



import com.museum.model.*;

import com.museum.repository.*;

import org.springframework.stereotype.Service;

import org.springframework.transaction.annotation.Isolation;

import org.springframework.transaction.annotation.Transactional;



import java.time.LocalDate;

import java.time.LocalTime;

import java.util.LinkedHashMap;

import java.util.List;

import java.util.Map;



@Service

public class BookingService {



    private final BookingRepository bookingRepository;

    private final VisitorRepository visitorRepository;

    private final ExhibitRepository exhibitRepository;

    private final ScheduleRepository scheduleRepository;

    private final TicketRepository ticketRepository;
```

```java
    private final TicketPricingService pricingService;

    private final NotificationManager notificationManager;



    public BookingService(BookingRepository bookingRepository,

                          VisitorRepository visitorRepository,

                          ExhibitRepository exhibitRepository,

                          ScheduleRepository scheduleRepository,

                          TicketRepository ticketRepository,

                          TicketPricingService pricingService,

                          NotificationManager notificationManager) {

        this.bookingRepository = bookingRepository;

        this.visitorRepository = visitorRepository;

        this.exhibitRepository = exhibitRepository;

        this.scheduleRepository = scheduleRepository;

        this.ticketRepository = ticketRepository;

        this.pricingService = pricingService;

        this.notificationManager = notificationManager;

    }



    @Transactional(isolation = Isolation.SERIALIZABLE)

    public Map<String, Object> createBooking(Long visitorId, Long exhibitId,

                                             LocalDate visitDate, LocalTime visitTime,

                                             TicketType ticketType, int numberOfTickets) {



        if (numberOfTickets < 1)

            throw new IllegalArgumentException("Number of tickets must be at least 1.");

        if (numberOfTickets > 50)

            throw new IllegalArgumentException("Cannot book more than 50 tickets per booking.");



        Visitor visitor = visitorRepository.findById(visitorId)

                .orElseThrow(() -> new RuntimeException("Visitor not found: " + visitorId));
```

```java
        Exhibit exhibit = exhibitRepository.findById(exhibitId)
                .orElseThrow(() -> new RuntimeException("Exhibit not found: " + exhibitId));


        int totalAlreadyBooked = bookingRepository.countConfirmedTicketsForExhibitDate(exhibitId,
visitDate);
        int totalRequestedTickets = totalAlreadyBooked + numberOfTickets;


        if (totalAlreadyBooked >= 50)
            throw new RuntimeException(
                    "This exhibit is fully booked on " + visitDate + ". Maximum 50 tickets per day. " +
                    "Currently: " + totalAlreadyBooked + " tickets booked.");


        if (totalRequestedTickets > 50)
            throw new RuntimeException(
                    "Cannot accommodate " + numberOfTickets + " tickets. " +
                    "Currently booked: " + totalAlreadyBooked + " / 50. " +
                    "Only " + (50 - totalAlreadyBooked) + " ticket(s) remaining on " + visitDate + ".");


        Schedule schedule = new Schedule();
        schedule.setExhibit(exhibit);
        schedule.setRequestedBy(visitor);
        schedule.setVisitDate(visitDate);
        schedule.setStartTime(visitTime);
        schedule.setEndTime(visitTime.plusHours(1));
        schedule.setMaxVisitors(numberOfTickets);
        schedule.setCurrentVisitors(numberOfTickets);
        schedule.setStatus(ScheduleStatus.PENDING);
        schedule = scheduleRepository.save(schedule);


        Booking booking = new Booking();
        booking.setVisitor(visitor);
```

```java
        booking.setSchedule(schedule);

        booking.setStatus("PENDING");

        booking.setNumberOfTickets(numberOfTickets);

        booking = bookingRepository.save(booking);



        TicketType type = ticketType != null ? ticketType : TicketType.ADULT;

        double unitPrice = pricingService.getPrice(type);

        Ticket ticket = new Ticket();

        ticket.setBooking(booking);

        ticket.setTicketType(type);

        ticket.setPrice(unitPrice * numberOfTickets);

        ticket.setPaymentStatus("PENDING");

        ticket = ticketRepository.save(ticket);



        notificationManager.notifyUser(visitor,

                "Your booking request for '" + exhibit.getName() +

                "' on " + visitDate + " at " + visitTime +

                " (" + numberOfTickets + " ticket(s) × ₹" + unitPrice +

                " = ₹" + ticket.getPrice() + ") has been submitted. Awaiting admin confirmation.");



        return buildFlatWithTickets(booking);

    }



    @Transactional(isolation = Isolation.SERIALIZABLE)

    public Map<String, Object> createBookingWithMultipleTypes(Long visitorId, Long exhibitId,

                                                              LocalDate visitDate, LocalTime visitTime,

                                                              Map<String, Object> ticketBreakdown) {



        int totalTickets = 0;

        for (Object value : ticketBreakdown.values()) {

            int qty = Integer.parseInt(value.toString());
```

```
            if (qty < 0)
                throw new IllegalArgumentException("Ticket quantities cannot be negative.");
            totalTickets += qty;
        }

        if (totalTickets < 1)
            throw new IllegalArgumentException("Must book at least 1 ticket.");
        if (totalTickets > 50)
            throw new IllegalArgumentException("Cannot book more than 50 tickets per booking.");

        Visitor visitor = visitorRepository.findById(visitorId)
                .orElseThrow(() -> new RuntimeException("Visitor not found: " + visitorId));
        Exhibit exhibit = exhibitRepository.findById(exhibitId)
                .orElseThrow(() -> new RuntimeException("Exhibit not found: " + exhibitId));

        int totalAlreadyBooked = bookingRepository.countConfirmedTicketsForExhibitDate(exhibitId,
visitDate);
        int totalRequestedTickets = totalAlreadyBooked + totalTickets;

        if (totalAlreadyBooked >= 50)
            throw new RuntimeException(
                    "This exhibit is fully booked on " + visitDate + ". Maximum 50 tickets per day. " +
                    "Currently: " + totalAlreadyBooked + " tickets booked.");

        if (totalRequestedTickets > 50)
            throw new RuntimeException(
                    "Cannot accommodate " + totalTickets + " tickets. " +
                    "Currently booked: " + totalAlreadyBooked + " / 50. " +
                    "Only " + (50 - totalAlreadyBooked) + " ticket(s) remaining on " + visitDate + ".");

        Schedule schedule = new Schedule();
```

```
        schedule.setExhibit(exhibit);

        schedule.setRequestedBy(visitor);

        schedule.setVisitDate(visitDate);

        schedule.setStartTime(visitTime);

        schedule.setEndTime(visitTime.plusHours(1));

        schedule.setMaxVisitors(totalTickets);

        schedule.setCurrentVisitors(totalTickets);

        schedule.setStatus(ScheduleStatus.PENDING);

        schedule = scheduleRepository.save(schedule);



        Booking booking = new Booking();

        booking.setVisitor(visitor);

        booking.setSchedule(schedule);

        booking.setStatus("PENDING");

        booking.setNumberOfTickets(totalTickets);

        booking = bookingRepository.save(booking);



        double totalPrice = 0;

        StringBuilder ticketSummary = new StringBuilder();

        for (Map.Entry<String, Object> entry : ticketBreakdown.entrySet()) {

            int qty = Integer.parseInt(entry.getValue().toString());

            if (qty <= 0) continue;



            try {

                TicketType type = TicketType.valueOf(entry.getKey().toUpperCase());

                double unitPrice = pricingService.getPrice(type);



                Ticket ticket = new Ticket();

                ticket.setBooking(booking);

                ticket.setTicketType(type);

                ticket.setPrice(unitPrice * qty);
```

```
                ticket.setPaymentStatus("PENDING");
                ticketRepository.save(ticket);

                totalPrice += ticket.getPrice();
                if (ticketSummary.length() > 0) ticketSummary.append(", ");
                ticketSummary.append(qty).append("x ").append(type).append(" @ ₹").append(unitPrice);
            } catch (IllegalArgumentException e) {
            }
        }

        notificationManager.notifyUser(visitor,
                "Your booking request for '" + exhibit.getName() +
                "' on " + visitDate + " at " + visitTime +
                " (" + ticketSummary + " = ₹" + totalPrice + ")" +
                " has been submitted. Awaiting admin confirmation.");

        return buildFlatWithTickets(booking);
    }

    @Transactional
    public Map<String, Object> confirmBooking(Long bookingId) {
        Booking booking = getBookingById(bookingId);
        if (!"PENDING".equals(booking.getStatus()))
            throw new RuntimeException("Booking is not PENDING.");

        booking.setStatus("CONFIRMED");
        bookingRepository.save(booking);

        Schedule schedule = booking.getSchedule();
        schedule.setStatus(ScheduleStatus.APPROVED);
        scheduleRepository.save(schedule);
```

```java
    List<Ticket> tickets = ticketRepository.findByBooking_BookingId(bookingId);
    if (tickets.isEmpty())
        throw new RuntimeException("No tickets found for booking: " + bookingId);

    double totalPrice = 0;
    for (Ticket ticket : tickets) {
        ticket.setPaymentStatus("PAID");
        ticketRepository.save(ticket);
        totalPrice += ticket.getPrice();
    }

    notificationManager.notifyUser(booking.getVisitor(),
            "Your booking for '" + schedule.getExhibit().getName() +
            "' on " + schedule.getVisitDate() + " is CONFIRMED! " +
            booking.getNumberOfTickets() + " ticket(s) | ₹" + totalPrice);

    return buildFlatWithTickets(booking);
}

@Transactional
public Map<String, Object> rejectBooking(Long bookingId) {
    Booking booking = getBookingById(bookingId);
    if (!"PENDING".equals(booking.getStatus()))
        throw new RuntimeException("Booking is not PENDING.");

    booking.setStatus("CANCELLED");
    bookingRepository.save(booking);

    Schedule schedule = booking.getSchedule();
    schedule.setStatus(ScheduleStatus.REJECTED);
```

```
        scheduleRepository.save(schedule);


        List<Ticket> tickets = ticketRepository.findByBooking_BookingId(bookingId);
        for (Ticket ticket : tickets) {
            ticket.setPaymentStatus("REFUNDED");
            ticketRepository.save(ticket);
        }


        notificationManager.notifyUser(booking.getVisitor(),
                "Your booking for '" + schedule.getExhibit().getName() +
                "' on " + schedule.getVisitDate() + " was not approved.");


        return buildFlatWithTickets(booking);
    }


    @Transactional
    public void markAssigned(Long bookingId, String guideName) {
        Booking booking = getBookingById(bookingId);
        booking.setStatus("ASSIGNED");
        bookingRepository.save(booking);


        List<Ticket> tickets = ticketRepository.findByBooking_BookingId(bookingId);
        for (Ticket ticket : tickets) {
            ticket.setAssignedGuideName(guideName);
            ticketRepository.save(ticket);
        }
        notificationManager.notifyUser(booking.getVisitor(),
                "🗺️ Guide " + guideName + " has been assigned to your visit at '" +
                booking.getSchedule().getExhibit().getName() + "'.");
    }
```

```java
@Transactional
public Map<String, Object> cancelBooking(Long bookingId) {
    Booking booking = getBookingById(bookingId);
    if ("CANCELLED".equals(booking.getStatus()))
        throw new RuntimeException("Already cancelled.");

    booking.setStatus("CANCELLED");
    bookingRepository.save(booking);

    List<Ticket> tickets = ticketRepository.findByBooking_BookingId(bookingId);
    for (Ticket ticket : tickets) {
        ticket.setPaymentStatus("REFUNDED");
        ticketRepository.save(ticket);
    }

    notificationManager.notifyUser(booking.getVisitor(), "Your booking has been cancelled.");
    return buildFlatWithTickets(booking);
}

@Transactional(readOnly = true)
public List<Map<String, Object>> getAllBookingsFlat() {
    return bookingRepository.findAll().stream().map(this::buildFlatWithTickets).toList();
}

@Transactional(readOnly = true)
public List<Map<String, Object>> getPendingBookingsFlat() {
    return bookingRepository.findByStatus("PENDING").stream().map(this::buildFlatWithTickets).toList();
}

@Transactional(readOnly = true)
public List<Map<String, Object>> getBookingsByVisitorFlat(Long visitorId) {
```

```
        return
bookingRepository.findByVisitor_UserId(visitorId).stream().map(this::buildFlatWithTickets).toList();

    }



    @Transactional(readOnly = true)

    public Booking getBookingById(Long id) {

        return bookingRepository.findById(id)

                .orElseThrow(() -> new RuntimeException("Booking not found: " + id));

    }



    @Transactional(readOnly = true)

    public Map<String, Object> getBookingFlat(Long id) {

        return buildFlatWithTickets(getBookingById(id));

    }



    private Map<String, Object> buildFlat(Booking b) {

        Ticket ticket = null;

        try { ticket =
ticketRepository.findByBooking_BookingId(b.getBookingId()).stream().findFirst().orElse(null); }

        catch (Exception ignored) {}

        Schedule s = b.getSchedule();

        Exhibit e = s != null ? s.getExhibit() : null;

        String guideName = ticket != null ? ticket.getAssignedGuideName() : null;

        return flattenBooking(b, ticket, s, e, guideName);

    }



    private Map<String, Object> buildFlatWithTickets(Booking b) {

        List<Ticket> tickets = ticketRepository.findByBooking_BookingId(b.getBookingId());

        Schedule s = b.getSchedule();

        Exhibit e = s != null ? s.getExhibit() : null;



        Map<String, Object> m = new LinkedHashMap<>();
```

```
        m.put("bookingId", b.getBookingId());

        m.put("id", b.getBookingId());

        m.put("status", b.getStatus());

        m.put("bookingDate", b.getBookingDate() != null ? b.getBookingDate().toString() : "");

        m.put("numberOfTickets", b.getNumberOfTickets());



        m.put("exhibitName", e != null ? e.getName() : "N/A");

        m.put("exhibitId", e != null ? e.getExhibitId() : 0);

        m.put("category", e != null ? e.getCategory() : "");



        m.put("visitDate", s != null && s.getVisitDate() != null ? s.getVisitDate().toString() : "");

        m.put("visitTime", s != null && s.getStartTime() != null ? s.getStartTime().toString() : "");

        m.put("scheduleId", s != null ? s.getScheduleId() : 0);

        m.put("maxVisitors", s != null ? s.getMaxVisitors() : 0);



        m.put("visitorName", b.getVisitor() != null ? b.getVisitor().getName() : "N/A");

        m.put("visitorEmail", b.getVisitor() != null ? b.getVisitor().getEmail() : "");

        m.put("visitorId", b.getVisitor() != null ? b.getVisitor().getUserId() : 0);



        List<Map<String, Object>> ticketList = new java.util.ArrayList<>();

        double totalPrice = 0;

        String assignedGuide = "Not assigned";



        for (Ticket ticket : tickets) {

            Map<String, Object> ticketInfo = new LinkedHashMap<>();

            ticketInfo.put("ticketId", ticket.getTicketId());

            ticketInfo.put("ticketType", ticket.getTicketType() != null ? ticket.getTicketType().name() :
"ADULT");

            ticketInfo.put("price", ticket.getPrice());

            ticketInfo.put("paymentStatus", ticket.getPaymentStatus());

            ticketInfo.put("purchaseDate", ticket.getPurchaseDate() != null ?
ticket.getPurchaseDate().toString() : "");
```

```java
            ticketList.add(ticketInfo);
            totalPrice += ticket.getPrice();
            if (ticket.getAssignedGuideName() != null) {
                assignedGuide = ticket.getAssignedGuideName();
            }
        }

        if (ticketList.isEmpty()) {
            m.put("tickets", java.util.Collections.emptyList());
            m.put("totalPrice", 0);
        } else {
            m.put("tickets", ticketList);
            m.put("totalPrice", totalPrice);
            m.put("ticketId", ticketList.get(0).get("ticketId"));
            m.put("ticketType", ticketList.get(0).get("ticketType"));
        }

        m.put("assignedGuide", assignedGuide);
        return m;
    }

    private Map<String, Object> flattenBooking(Booking b, Ticket ticket,
                                               Schedule schedule, Exhibit exhibit,
                                               String guideName) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("bookingId", b.getBookingId());
        m.put("id", b.getBookingId());
        m.put("status", b.getStatus());
        m.put("bookingDate", b.getBookingDate() != null ? b.getBookingDate().toString() : "");
        m.put("numberOfTickets", b.getNumberOfTickets());
```

```
        m.put("exhibitName", exhibit != null ? exhibit.getName() : "N/A");

        m.put("exhibitId", exhibit != null ? exhibit.getExhibitId() : 0);

        m.put("category", exhibit != null ? exhibit.getCategory() : "");



        m.put("visitDate", schedule != null && schedule.getVisitDate() != null ?
schedule.getVisitDate().toString() : "");

        m.put("visitTime", schedule != null && schedule.getStartTime() != null ?
schedule.getStartTime().toString() : "");

        m.put("scheduleId", schedule != null ? schedule.getScheduleId() : 0);

        m.put("maxVisitors", schedule != null ? schedule.getMaxVisitors() : 0);



        m.put("visitorName", b.getVisitor() != null ? b.getVisitor().getName() : "N/A");

        m.put("visitorEmail", b.getVisitor() != null ? b.getVisitor().getEmail() : "");

        m.put("visitorId", b.getVisitor() != null ? b.getVisitor().getUserId() : 0);



        if (ticket != null) {

            m.put("ticketId", ticket.getTicketId());

            m.put("ticketType", ticket.getTicketType() != null ? ticket.getTicketType().name() : "ADULT");

            m.put("price", ticket.getPrice());

            m.put("paymentStatus", ticket.getPaymentStatus());

            m.put("purchaseDate", ticket.getPurchaseDate() != null ? ticket.getPurchaseDate().toString() :
"");

        } else {

            m.put("ticketId", null);

            m.put("ticketType", "N/A");

            m.put("price", 0);

            m.put("paymentStatus", "N/A");

            m.put("purchaseDate", "");

        }



        m.put("assignedGuide", guideName != null ? guideName : "Not assigned");

        return m;
```

```
    }

}
```

- ScheduleService.java

```
package com.museum.service;


import com.museum.dto.ScheduleRequestDTO;

import com.museum.model.*;

import com.museum.repository.ExhibitRepository;

import com.museum.repository.ScheduleRepository;

import com.museum.repository.VisitorRepository;

import org.springframework.stereotype.Service;

import org.springframework.transaction.annotation.Transactional;


import java.time.LocalDate;

import java.util.List;


@Service

public class ScheduleService {


    private final ScheduleRepository scheduleRepository;

    private final ExhibitRepository exhibitRepository;

    private final VisitorRepository visitorRepository;

    private final NotificationManager notificationManager;


    public ScheduleService(ScheduleRepository scheduleRepository,

                           ExhibitRepository exhibitRepository,

                           VisitorRepository visitorRepository,

                           NotificationManager notificationManager) {

        this.scheduleRepository = scheduleRepository;

        this.exhibitRepository = exhibitRepository;
```

```java
        this.visitorRepository = visitorRepository;

        this.notificationManager = notificationManager;

    }



    @Transactional

    public Schedule requestSchedule(ScheduleRequestDTO dto) {

        Exhibit exhibit = exhibitRepository.findById(dto.getExhibitId())

                .orElseThrow(() -> new RuntimeException("Exhibit not found: " + dto.getExhibitId()));

        Visitor visitor = visitorRepository.findById(dto.getVisitorId())

                .orElseThrow(() -> new RuntimeException("Visitor not found: " + dto.getVisitorId()));



        Schedule schedule = new Schedule();

        schedule.setExhibit(exhibit);

        schedule.setRequestedBy(visitor);

        schedule.setVisitDate(dto.getVisitDate());

        schedule.setStartTime(dto.getResolvedStartTime());

        schedule.setEndTime(dto.getResolvedEndTime());

        schedule.setMaxVisitors(dto.getMaxVisitors() > 0 ? dto.getMaxVisitors() : 30);

        schedule.setStatus(ScheduleStatus.PENDING);



        Schedule saved = scheduleRepository.save(schedule);



        notificationManager.notifyUser(visitor,

                "Your visit request for exhibit '" + exhibit.getName() +

                "' on " + dto.getVisitDate() + " at " + schedule.getStartTime() +

                " has been submitted and is pending approval.");



        return saved;

    }



    @Transactional
```

```java
public Schedule approveSchedule(Long scheduleId) {

    Schedule schedule = getScheduleById(scheduleId);

    if (schedule.getStatus() != ScheduleStatus.PENDING)

        throw new RuntimeException("Schedule is not PENDING.");

    schedule.setStatus(ScheduleStatus.APPROVED);

    Schedule approved = scheduleRepository.save(schedule);

    if (schedule.getRequestedBy() != null) {

        notificationManager.notifyUser(schedule.getRequestedBy(),

                "Your visit to '" + schedule.getExhibit().getName() +

                "' on " + schedule.getVisitDate() + " at " + schedule.getStartTime() +

                " has been APPROVED!");

    }

    return approved;

}



@Transactional

public Schedule rejectSchedule(Long scheduleId) {

    Schedule schedule = getScheduleById(scheduleId);

    if (schedule.getStatus() != ScheduleStatus.PENDING)

        throw new RuntimeException("Schedule is not PENDING.");

    schedule.setStatus(ScheduleStatus.REJECTED);

    Schedule rejected = scheduleRepository.save(schedule);

    if (schedule.getRequestedBy() != null) {

        notificationManager.notifyUser(schedule.getRequestedBy(),

                "Your visit request for '" + schedule.getExhibit().getName() +

                "' on " + schedule.getVisitDate() + " has been REJECTED.");

    }

    return rejected;

}



@Transactional(readOnly = true)
```

```java
public List<Schedule> checkAvailability(Long exhibitId, LocalDate date) {

    return scheduleRepository.findAvailableSlots(exhibitId, date)

            .stream().filter(Schedule::hasAvailableSlots).toList();

}



@Transactional(readOnly = true)

public List<Schedule> getAllSchedules() {

    return scheduleRepository.findAll();

}



@Transactional(readOnly = true)

public List<Schedule> getPendingSchedules() {

    return scheduleRepository.findByStatus(ScheduleStatus.PENDING);

}



@Transactional(readOnly = true)

public List<Schedule> getSchedulesByExhibit(Long exhibitId) {

    return scheduleRepository.findByExhibit_ExhibitId(exhibitId);

}



@Transactional(readOnly = true)

public List<Schedule> getSchedulesByVisitor(Long visitorId) {

    Visitor visitor = visitorRepository.findById(visitorId)

            .orElseThrow(() -> new RuntimeException("Visitor not found: " + visitorId));

    return scheduleRepository.findByRequestedBy(visitor);

}



@Transactional(readOnly = true)

public Schedule getScheduleById(Long id) {

    return scheduleRepository.findById(id)

            .orElseThrow(() -> new RuntimeException("Schedule not found: " + id));
```

```
    }

}
```

4.4 Repository

- BookingRepository.java

```
package com.museum.repository;


import com.museum.model.Booking;

import com.museum.model.Visitor;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.jpa.repository.Query;

import org.springframework.data.repository.query.Param;

import org.springframework.stereotype.Repository;


import java.time.LocalDate;

import java.util.List;


@Repository

public interface BookingRepository extends JpaRepository<Booking, Long> {


    List<Booking> findByVisitor(Visitor visitor);

    List<Booking> findByVisitor_UserId(Long visitorId);

    List<Booking> findByStatus(String status);


    long countBySchedule_ScheduleIdAndStatusNot(Long scheduleId, String status);


    @Query("SELECT COALESCE(SUM(b.numberOfTickets), 0) FROM Booking b " +

            "WHERE b.schedule.scheduleId = :scheduleId AND b.status <> 'CANCELLED'")

    int sumTicketsBySchedule(@Param("scheduleId") Long scheduleId);


    @Query("SELECT COALESCE(SUM(b.numberOfTickets), 0) FROM Booking b " +
```

```
            "INNER JOIN b.schedule s " +

            "WHERE s.exhibit.exhibitId = :exhibitId AND s.visitDate = :visitDate " +

            "AND b.status IN ('CONFIRMED', 'ASSIGNED')")

    int countConfirmedTicketsForExhibitDate(@Param("exhibitId") Long exhibitId, @Param("visitDate")
LocalDate visitDate);

}
```

- ScheduleRepository.java

```
package com.museum.repository;



import com.museum.model.Schedule;

import com.museum.model.ScheduleStatus;

import com.museum.model.Visitor;

import jakarta.persistence.LockModeType;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.jpa.repository.Lock;

import org.springframework.data.jpa.repository.Query;

import org.springframework.data.repository.query.Param;

import org.springframework.stereotype.Repository;



import java.time.LocalDate;

import java.util.List;

import java.util.Optional;



@Repository

public interface ScheduleRepository extends JpaRepository<Schedule, Long> {



    List<Schedule> findByStatus(ScheduleStatus status);

    List<Schedule> findByExhibit_ExhibitId(Long exhibitId);

    List<Schedule> findByRequestedBy(Visitor visitor);

    List<Schedule> findByVisitDate(LocalDate date);
```

```
    @Query("SELECT s FROM Schedule s WHERE s.exhibit.exhibitId = :exhibitId " +
           "AND s.visitDate = :date AND s.status = 'APPROVED'")
    List<Schedule> findAvailableSlots(@Param("exhibitId") Long exhibitId,
                                      @Param("date") LocalDate date);

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT s FROM Schedule s WHERE s.scheduleId = :id")
    Optional<Schedule> findByIdWithLock(@Param("id") Long id);
}
```

4.5 DTO

- BookingDTO.java

```
package com.museum.dto;

import com.museum.model.TicketType;
import jakarta.validation.constraints.Min;

public class BookingDTO {
    private Long visitorId;
    private Long scheduleId;
    private TicketType ticketType;

    @Min(value = 1, message = "Number of tickets must be at least 1")
    private int numberOfTickets = 1;

    public Long getVisitorId() { return visitorId; }
    public void setVisitorId(Long v) { this.visitorId = v; }
    public Long getScheduleId() { return scheduleId; }
    public void setScheduleId(Long v) { this.scheduleId = v; }
    public TicketType getTicketType() { return ticketType; }
```

```
    public void setTicketType(TicketType v) { this.ticketType = v; }

    public int getNumberOfTickets() { return numberOfTickets; }

    public void setNumberOfTickets(int v) { this.numberOfTickets = v; }

}
```

- ScheduleRequestDTO.java

```
package com.museum.dto;


import jakarta.validation.constraints.NotNull;

import java.time.LocalDate;

import java.time.LocalTime;


public class ScheduleRequestDTO {


    @NotNull(message = "Exhibit ID is required")

    private Long exhibitId;


    @NotNull(message = "Visitor ID is required")

    private Long visitorId;


    @NotNull(message = "Visit date is required")

    private LocalDate visitDate;


    private LocalTime startTime;

    private LocalTime endTime;

    private String visitTime;

    private int maxVisitors = 30;


    public LocalTime getResolvedStartTime() {

        if (startTime != null) return startTime;

        if (visitTime != null && !visitTime.isBlank()) {
```

```
            try { return LocalTime.parse(visitTime); } catch (Exception ignored) {}

        }

        return LocalTime.of(10, 0);

    }



    public LocalTime getResolvedEndTime() {

        if (endTime != null) return endTime;

        return getResolvedStartTime().plusHours(1);

    }



    public Long getExhibitId() { return exhibitId; }

    public void setExhibitId(Long exhibitId) { this.exhibitId = exhibitId; }

    public Long getVisitorId() { return visitorId; }

    public void setVisitorId(Long visitorId) { this.visitorId = visitorId; }

    public LocalDate getVisitDate() { return visitDate; }

    public void setVisitDate(LocalDate visitDate) { this.visitDate = visitDate; }

    public LocalTime getStartTime() { return startTime; }

    public void setStartTime(LocalTime startTime) { this.startTime = startTime; }

    public LocalTime getEndTime() { return endTime; }

    public void setEndTime(LocalTime endTime) { this.endTime = endTime; }

    public String getVisitTime() { return visitTime; }

    public void setVisitTime(String visitTime) { this.visitTime = visitTime; }

    public int getMaxVisitors() { return maxVisitors; }

    public void setMaxVisitors(int maxVisitors) { this.maxVisitors = maxVisitors; }

}
```

5. Ticket Module

5.1 Model

- Ticket.java

```
package com.museum.model;



import jakarta.persistence.*;
```

```java
import java.time.LocalDateTime;



@Entity

@Table(name = "tickets")

public class Ticket {



    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long ticketId;



    @Column(nullable = false)

    private double price;



    @Column(nullable = false, updatable = false)

    private LocalDateTime purchaseDate;



    @Enumerated(EnumType.STRING)

    @Column(nullable = false)

    private TicketType ticketType;



    @Column(nullable = false)

    private String paymentStatus = "PENDING";



    private String assignedGuideName;



    @ManyToOne(fetch = FetchType.LAZY)

    @JoinColumn(name = "booking_id", nullable = false)

    private Booking booking;



    @PrePersist

    protected void prePersist() { this.purchaseDate = LocalDateTime.now(); }
```

```
    public Long getTicketId() { return ticketId; }

    public void setTicketId(Long ticketId) { this.ticketId = ticketId; }

    public double getPrice() { return price; }

    public void setPrice(double price) { this.price = price; }

    public LocalDateTime getPurchaseDate() { return purchaseDate; }

    public void setPurchaseDate(LocalDateTime purchaseDate) { this.purchaseDate = purchaseDate; }

    public TicketType getTicketType() { return ticketType; }

    public void setTicketType(TicketType ticketType) { this.ticketType = ticketType; }

    public String getPaymentStatus() { return paymentStatus; }

    public void setPaymentStatus(String paymentStatus) { this.paymentStatus = paymentStatus; }

    public String getAssignedGuideName() { return assignedGuideName; }

    public void setAssignedGuideName(String assignedGuideName) { this.assignedGuideName =
assignedGuideName; }

    public Booking getBooking() { return booking; }

    public void setBooking(Booking booking) { this.booking = booking; }

}
```

- TicketType.java

```
package com.museum.model;



public enum TicketType {

    ADULT,

    CHILD,

    STUDENT,

    VIP

}
```

5.2 Controller

- TicketController.java

```java
package com.museum.controller;


import com.museum.model.Ticket;

import com.museum.model.TicketType;

import com.museum.service.TicketService;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;


import java.util.List;

import java.util.Map;


@RestController

@RequestMapping("/api/tickets")

public class TicketController {


    private final TicketService ticketService;


    public TicketController(TicketService ticketService) {

        this.ticketService = ticketService;

    }


    @GetMapping({"", "/"})

    public ResponseEntity<List<Ticket>> getAllTickets() { return
ResponseEntity.ok(ticketService.getAllTickets()); }


    @GetMapping("/{id}")

    public ResponseEntity<?> getById(@PathVariable Long id) {

        try { return ResponseEntity.ok(ticketService.getTicketById(id)); }

        catch (RuntimeException e) { return ResponseEntity.notFound().build(); }

    }
```

```
    @GetMapping("/booking/{bookingId}")

    public ResponseEntity<?> getByBooking(@PathVariable Long bookingId) {

        try { return ResponseEntity.ok(ticketService.getTicketByBookingId(bookingId)); }

        catch (RuntimeException e) { return ResponseEntity.notFound().build(); }

    }



    @GetMapping("/type/{type}")

    public ResponseEntity<List<Ticket>> getByType(@PathVariable String type) {

        return ResponseEntity.ok(ticketService.getTicketsByType(TicketType.valueOf(type.toUpperCase())));

    }



    @GetMapping("/revenue")

    public ResponseEntity<Map<String, Object>> getRevenue() {

        return ResponseEntity.ok(Map.of("totalRevenue", ticketService.getTotalRevenue(), "currency",
"INR"));

    }

}
```

5.3 Service

- TicketService.java

```
package com.museum.service;



import com.museum.model.Ticket;

import com.museum.model.TicketType;

import com.museum.repository.TicketRepository;

import org.springframework.stereotype.Service;

import org.springframework.transaction.annotation.Transactional;

import java.util.List;



@Service
```

```java
public class TicketService {

    private final TicketRepository ticketRepository;

    public TicketService(TicketRepository ticketRepository) { this.ticketRepository = ticketRepository; }



    @Transactional(readOnly = true) public List<Ticket> getAllTickets() { return
ticketRepository.findAll(); }

    @Transactional(readOnly = true) public Ticket getTicketById(Long id) {

        return ticketRepository.findById(id).orElseThrow(() -> new RuntimeException("Ticket not found: " +
id));

    }

    @Transactional(readOnly = true) public Ticket getTicketByBookingId(Long bookingId) {

        List<Ticket> tickets = ticketRepository.findByBooking_BookingId(bookingId);

        if (tickets.isEmpty())

            throw new RuntimeException("Ticket not found for booking: " + bookingId);

        return tickets.get(0);

    }

    @Transactional(readOnly = true) public List<Ticket> getTicketsByType(TicketType type) { return
ticketRepository.findByTicketType(type); }

    @Transactional(readOnly = true) public double getTotalRevenue() {

        Double r = ticketRepository.totalRevenue(); return r != null ? r : 0.0;

    }

}
```

- TicketPricingService.java

```java
package com.museum.service;



import com.museum.model.TicketType;

import org.springframework.stereotype.Service;



@Service

public class TicketPricingService {
```

```
    public double getPrice(TicketType type) {

        return switch (type) {

            case ADULT   -> 500.00;

            case CHILD   -> 200.00;

            case STUDENT -> 150.00;

            case VIP     -> 1500.00;

        };

    }

}
```

5.4 Repository

- TicketRepository.java

```
package com.museum.repository;



import com.museum.model.Ticket;

import com.museum.model.TicketType;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.jpa.repository.Query;

import org.springframework.stereotype.Repository;



import java.util.List;

import java.util.Optional;



@Repository

public interface TicketRepository extends JpaRepository<Ticket, Long> {



    List<Ticket> findByBooking_BookingId(Long bookingId);



    List<Ticket> findByTicketType(TicketType type);
```

```
    @Query("SELECT SUM(t.price) FROM Ticket t WHERE t.paymentStatus = 'PAID'")

    Double totalRevenue();

}
```

5.5 DTO

- TicketDTO.java

```
package com.museum.dto;


import com.museum.model.TicketType;


import java.time.LocalDateTime;


public class TicketDTO {


    private Long ticketId;

    private Long bookingId;

    private TicketType ticketType;

    private double price;

    private String paymentStatus;

    private LocalDateTime purchaseDate;


    private String visitorName;

    private String visitorEmail;


    private String exhibitName;

    private String visitDate;

    private String timeSlot;


    public TicketDTO() {

    }
```

```java
    public TicketDTO(Long ticketId, Long bookingId, TicketType ticketType, double price, String
paymentStatus,
                     LocalDateTime purchaseDate, String visitorName, String visitorEmail, String
exhibitName,
                     String visitDate, String timeSlot) {
        this.ticketId = ticketId;
        this.bookingId = bookingId;
        this.ticketType = ticketType;
        this.price = price;
        this.paymentStatus = paymentStatus;
        this.purchaseDate = purchaseDate;
        this.visitorName = visitorName;
        this.visitorEmail = visitorEmail;
        this.exhibitName = exhibitName;
        this.visitDate = visitDate;
        this.timeSlot = timeSlot;
    }

    public Long getTicketId() {
        return ticketId;
    }

    public void setTicketId(Long ticketId) {
        this.ticketId = ticketId;
    }

    public Long getBookingId() {
        return bookingId;
    }

    public void setBookingId(Long bookingId) {
```

```java
        this.bookingId = bookingId;
    }

    public TicketType getTicketType() {
        return ticketType;
    }

    public void setTicketType(TicketType ticketType) {
        this.ticketType = ticketType;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public String getPaymentStatus() {
        return paymentStatus;
    }

    public void setPaymentStatus(String paymentStatus) {
        this.paymentStatus = paymentStatus;
    }

    public LocalDateTime getPurchaseDate() {
        return purchaseDate;
    }
```

```java
    public void setPurchaseDate(LocalDateTime purchaseDate) {

        this.purchaseDate = purchaseDate;

    }



    public String getVisitorName() {

        return visitorName;

    }



    public void setVisitorName(String visitorName) {

        this.visitorName = visitorName;

    }



    public String getVisitorEmail() {

        return visitorEmail;

    }



    public void setVisitorEmail(String visitorEmail) {

        this.visitorEmail = visitorEmail;

    }



    public String getExhibitName() {

        return exhibitName;

    }



    public void setExhibitName(String exhibitName) {

        this.exhibitName = exhibitName;

    }



    public String getVisitDate() {

        return visitDate;

    }
```

```
    public void setVisitDate(String visitDate) {
        this.visitDate = visitDate;
    }

    public String getTimeSlot() {
        return timeSlot;
    }

    public void setTimeSlot(String timeSlot) {
        this.timeSlot = timeSlot;
    }
}
```

6. Guide Module

6.1 Model

- Guide.java

```
package com.museum.model;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "guides")
@PrimaryKeyJoinColumn(name = "user_id")
public class Guide extends User {

    private String specialization;
```

```java
    @Column(nullable = false)
    private boolean available = true;


    @JsonIgnore
    @OneToMany(mappedBy = "guide", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<GuideAssignment> assignments = new ArrayList<>();


    @PrePersist
    protected void prePersist() {
        this.setRole(UserRole.GUIDE);
    }


    public String getSpecialization() { return specialization; }
    public void setSpecialization(String specialization) { this.specialization = specialization; }
    public boolean isAvailable() { return available; }
    public void setAvailable(boolean available) { this.available = available; }
    public List<GuideAssignment> getAssignments() { return assignments; }
    public void setAssignments(List<GuideAssignment> assignments) { this.assignments = assignments; }
}
```

- GuideAssignment.java

```java
package com.museum.model;


import jakarta.persistence.*;
import java.time.LocalDateTime;


@Entity
@Table(name = "guide_assignments")
public class GuideAssignment {


    @Id
```

```java
    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long assignmentId;

    @Column(nullable = false, updatable = false)

    private LocalDateTime assignedDate;

    @Column(nullable = false)

    private String status = "PENDING";

    @ManyToOne(fetch = FetchType.LAZY)

    @JoinColumn(name = "guide_id", nullable = false)

    private Guide guide;

    @ManyToOne(fetch = FetchType.LAZY)

    @JoinColumn(name = "booking_id", nullable = false)

    private Booking booking;

    @ManyToOne(fetch = FetchType.LAZY)

    @JoinColumn(name = "exhibit_id", nullable = false)

    private Exhibit exhibit;

    @PrePersist

    protected void prePersist() { this.assignedDate = LocalDateTime.now(); }

    public Long getAssignmentId() { return assignmentId; }

    public void setAssignmentId(Long assignmentId) { this.assignmentId = assignmentId; }

    public LocalDateTime getAssignedDate() { return assignedDate; }

    public void setAssignedDate(LocalDateTime assignedDate) { this.assignedDate = assignedDate; }

    public String getStatus() { return status; }

    public void setStatus(String status) { this.status = status; }

    public Guide getGuide() { return guide; }

    public void setGuide(Guide guide) { this.guide = guide; }

    public Booking getBooking() { return booking; }

    public void setBooking(Booking booking) { this.booking = booking; }

    public Exhibit getExhibit() { return exhibit; }

    public void setExhibit(Exhibit exhibit) { this.exhibit = exhibit; }

}
```

6.2 Controller

- GuideController.java

```java
package com.museum.controller;



import com.museum.model.Guide;

import com.museum.model.Notification;

import com.museum.repository.GuideRepository;

import com.museum.service.GuideAssignmentService;

import com.museum.service.NotificationManager;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;


import java.util.List;

import java.util.Map;


@RestController

@RequestMapping("/api/guides")

public class GuideController {


    private final GuideRepository guideRepository;

    private final GuideAssignmentService assignmentService;

    private final NotificationManager notificationManager;


    public GuideController(GuideRepository guideRepository,

                           GuideAssignmentService assignmentService,

                           NotificationManager notificationManager) {

        this.guideRepository = guideRepository;

        this.assignmentService = assignmentService;

        this.notificationManager = notificationManager;

    }
```

```java
@GetMapping({"", "/"})
public ResponseEntity<?> getAllGuides() {
    List<Guide> guides = guideRepository.findAll();
    List<Map<String, Object>> result = guides.stream().map(g -> Map.<String, Object>of(
            "userId", g.getUserId(), "id", g.getUserId(), "name", g.getName(),
            "email", g.getEmail(),
            "specialization", g.getSpecialization() != null ? g.getSpecialization() : "",
            "available", g.isAvailable()
    )).toList();
    return ResponseEntity.ok(result);
}


@GetMapping("/{id}")
public ResponseEntity<?> getById(@PathVariable Long id) {
    return guideRepository.findById(id)
            .<ResponseEntity<?>>map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
}
@GetMapping("/{id}/assignments")
public ResponseEntity<?> getAssignments(@PathVariable Long id) {
    return ResponseEntity.ok(assignmentService.getAssignmentsByGuideFlat(id));
}
@PutMapping("/assignments/{assignmentId}/confirm")
public ResponseEntity<?> confirmAssignment(@PathVariable Long assignmentId) {
    try {
        return ResponseEntity.ok(assignmentService.confirmAssignment(assignmentId));
    } catch (RuntimeException e) {
        return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
    }
}
@PutMapping("/assignments/{assignmentId}/reject")
```

```
    public ResponseEntity<?> rejectAssignment(@PathVariable Long assignmentId) {

        try {

            return ResponseEntity.ok(assignmentService.rejectAssignment(assignmentId));

        } catch (RuntimeException e) {

            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));

        }

    }

    @GetMapping("/{id}/schedule")

    public ResponseEntity<?> getSchedule(@PathVariable Long id) {

        List<Map<String, Object>> all = assignmentService.getAssignmentsByGuideFlat(id);

        List<Map<String, Object>> confirmed = all.stream()

                .filter(a -> "CONFIRMED".equals(a.get("status"))).toList();

        return ResponseEntity.ok(Map.of("guideId", id, "confirmedAssignments", confirmed,

                "totalConfirmed", confirmed.size()));

    }

    @GetMapping("/{id}/notifications")

    public ResponseEntity<List<Notification>> getNotifications(@PathVariable Long id) {

        return ResponseEntity.ok(notificationManager.getNotificationsForUser(id));

    }

}
```

6.3 Service

- GuideAssignmentService.java

```
package com.museum.service;



import com.museum.model.*;

import com.museum.repository.*;

import org.springframework.stereotype.Service;

import org.springframework.transaction.annotation.Transactional;



import java.util.LinkedHashMap;
```

```java
import java.util.List;

import java.util.Map;



@Service

public class GuideAssignmentService {



    private final GuideAssignmentRepository assignmentRepository;

    private final GuideRepository guideRepository;

    private final BookingService bookingService;

    private final NotificationManager notificationManager;



    public GuideAssignmentService(GuideAssignmentRepository assignmentRepository,

                                  GuideRepository guideRepository,

                                  BookingService bookingService,

                                  NotificationManager notificationManager) {

        this.assignmentRepository = assignmentRepository;

        this.guideRepository = guideRepository;

        this.bookingService = bookingService;

        this.notificationManager = notificationManager;

    }



    @Transactional

    public Map<String, Object> assignGuideToBooking(Long guideId, Long bookingId) {

        Guide guide = guideRepository.findById(guideId)

                .orElseThrow(() -> new RuntimeException("Guide not found: " + guideId));

        Booking booking = bookingService.getBookingById(bookingId);



        if ("CANCELLED".equals(booking.getStatus()))

            throw new RuntimeException("Cannot assign guide to a cancelled booking.");

        if ("PENDING".equals(booking.getStatus()))

            throw new RuntimeException("Booking must be confirmed before assigning a guide.");
```

```java
    String conflictBooking = hasGuideConflict(guideId, booking);
    if (conflictBooking != null) {
        throw new RuntimeException("Guide is unavailable. " + conflictBooking);
    }

    GuideAssignment assignment = new GuideAssignment();
    assignment.setGuide(guide);
    assignment.setBooking(booking);
    assignment.setExhibit(booking.getSchedule().getExhibit());
    assignment.setStatus("PENDING");
    assignment = assignmentRepository.save(assignment);

    Schedule s = booking.getSchedule();
    String exhibitName = s != null && s.getExhibit() != null ? s.getExhibit().getName() : "an exhibit";
    String visitDate   = s != null && s.getVisitDate() != null ? s.getVisitDate().toString() : "TBD";
    String visitTime   = s != null && s.getStartTime() != null ? s.getStartTime().toString() : "TBD";
    String visitorName = booking.getVisitor() != null ? booking.getVisitor().getName() : "a visitor";

    notificationManager.notifyUser(guide,
            "You have been assigned to guide " + visitorName +
            " at '" + exhibitName + "' on " + visitDate + " at " + visitTime +
            ". Please confirm or reject.");

    return flattenAssignment(assignment, booking);
}

@Transactional
public Map<String, Object> confirmAssignment(Long assignmentId) {
    GuideAssignment assignment = getById(assignmentId);
    if (!"PENDING".equals(assignment.getStatus()))
```

```
        throw new RuntimeException("Assignment is not PENDING.");


    assignment.setStatus("CONFIRMED");
    assignmentRepository.save(assignment);


    bookingService.markAssigned(assignment.getBooking().getBookingId(),
                                assignment.getGuide().getName());


    return flattenAssignment(assignment, assignment.getBooking());
}


@Transactional
public Map<String, Object> rejectAssignment(Long assignmentId) {
    GuideAssignment assignment = getById(assignmentId);
    if (!"PENDING".equals(assignment.getStatus()))
        throw new RuntimeException("Assignment is not PENDING.");


    assignment.setStatus("REJECTED");
    assignmentRepository.save(assignment);


    notificationManager.notifyUser(assignment.getGuide(),
            "You have rejected the assignment for booking #" +
            assignment.getBooking().getBookingId() + ".");


    return flattenAssignment(assignment, assignment.getBooking());
}


@Transactional(readOnly = true)
public List<Map<String, Object>> getAllAssignmentsFlat() {
    return assignmentRepository.findAll().stream()
            .map(a -> flattenAssignment(a, a.getBooking())).toList();
```

```java
}


@Transactional(readOnly = true)
public List<Map<String, Object>> getAssignmentsByGuideFlat(Long guideId) {
    return assignmentRepository.findByGuide_UserId(guideId).stream()
            .map(a -> flattenAssignment(a, a.getBooking())).toList();
}


@Transactional(readOnly = true)
public List<Guide> getAllGuides() { return guideRepository.findAll(); }


@Transactional(readOnly = true)
public GuideAssignment getById(Long id) {
    return assignmentRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Assignment not found: " + id));
}


Map<String, Object> flattenAssignment(GuideAssignment a, Booking b) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("assignmentId",  a.getAssignmentId());
    m.put("id",            a.getAssignmentId());
    m.put("status",        a.getStatus());
    m.put("assignedDate",  a.getAssignedDate() != null ? a.getAssignedDate().toString() : "");
    m.put("guideName",     a.getGuide() != null ? a.getGuide().getName() : "N/A");
    m.put("guideId",       a.getGuide() != null ? a.getGuide().getUserId() : 0);
    if (b != null) {
        m.put("bookingId",    b.getBookingId());
        m.put("bookingStatus",b.getStatus());
        m.put("visitorName",  b.getVisitor() != null ? b.getVisitor().getName() : "N/A");
        m.put("visitorEmail", b.getVisitor() != null ? b.getVisitor().getEmail() : "");
        Schedule s = b.getSchedule();
```

```
            Exhibit  e = s != null ? s.getExhibit() : null;

            m.put("exhibitName", e != null ? e.getName() : "N/A");

            m.put("visitDate",   s != null && s.getVisitDate()  != null ? s.getVisitDate().toString()  :
"");

            m.put("visitTime",   s != null && s.getStartTime()  != null ? s.getStartTime().toString()  :
"");

        }

        return m;

    }



    private String hasGuideConflict(Long guideId, Booking newBooking) {

        Schedule newSchedule = newBooking.getSchedule();

        if (newSchedule == null || newSchedule.getVisitDate() == null) {

            return null;

        }



        List<GuideAssignment> confirmedAssignments =
assignmentRepository.findByGuide_UserId(guideId).stream()

                .filter(a -> "CONFIRMED".equals(a.getStatus()))

                .toList();



        for (GuideAssignment assignment : confirmedAssignments) {

            Booking existingBooking = assignment.getBooking();

            Schedule existingSchedule = existingBooking.getSchedule();



            if (existingSchedule == null || existingSchedule.getVisitDate() == null) {

                continue;

            }



            if (newSchedule.getVisitDate().equals(existingSchedule.getVisitDate())) {

                if (timesOverlap(newSchedule, existingSchedule)) {

                    return "Already assigned to booking #" + existingBooking.getBookingId() +
```

```
                            " at the same time (" + existingSchedule.getVisitDate() + " " +
                            existingSchedule.getStartTime() + "-" + existingSchedule.getEndTime() + ")";
                }
            }
        }

        return null;
    }

    private boolean timesOverlap(Schedule schedule1, Schedule schedule2) {
        java.time.LocalTime start1 = schedule1.getStartTime();
        java.time.LocalTime end1 = schedule1.getEndTime();
        java.time.LocalTime start2 = schedule2.getStartTime();
        java.time.LocalTime end2 = schedule2.getEndTime();

        if (start1 == null || start2 == null) {
            return true;
        }

        java.time.LocalTime actualEnd1 = end1 != null ? end1 : java.time.LocalTime.MAX;
        java.time.LocalTime actualEnd2 = end2 != null ? end2 : java.time.LocalTime.MAX;

        return start1.isBefore(actualEnd2) && start2.isBefore(actualEnd1);
    }
}
```

6.4 Repository

- GuideRepository.java

```
package com.museum.repository;

import com.museum.model.Guide;
```

```java
import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;


import java.util.List;

import java.util.Optional;


@Repository

public interface GuideRepository extends JpaRepository<Guide, Long> {


    Optional<Guide> findByEmail(String email);


    List<Guide> findByAvailable(boolean available);

}
```

- GuideAssignmentRepository.java

```java
package com.museum.repository;


import com.museum.model.GuideAssignment;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;

import java.util.List;

import java.util.Optional;


@Repository

public interface GuideAssignmentRepository extends JpaRepository<GuideAssignment, Long> {


    List<GuideAssignment> findByGuide_UserId(Long guideId);


    List<GuideAssignment> findByBooking_BookingId(Long bookingId);
```

```
    List<GuideAssignment> findByStatus(String status);


    Optional<GuideAssignment> findByBooking_BookingIdAndStatus(Long bookingId, String status);


    boolean existsByGuide_UserIdAndBooking_BookingId(Long guideId, Long bookingId);

}
```

6.5 DTO

- GuideAssignmentDTO.java

```
package com.museum.dto;


public class GuideAssignmentDTO {

    private Long guideId;

    private Long exhibitId;


    public Long getGuideId() { return guideId; }

    public void setGuideId(Long v) { this.guideId = v; }

    public Long getExhibitId() { return exhibitId; }

    public void setExhibitId(Long v) { this.exhibitId = v; }

}
```

7. Feedback Module

7.1 Model

- Feedback.java

```
package com.museum.model;


import jakarta.persistence.*;

import java.time.LocalDateTime;
```

```java
@Entity

@Table(name = "feedbacks")

public class Feedback {



    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long feedbackId;



    @Column(nullable = false)

    private int rating;



    @Column(columnDefinition = "TEXT")

    private String comments;



    @Column(nullable = false, updatable = false)

    private LocalDateTime feedbackDate;



    @ManyToOne(fetch = FetchType.LAZY)

    @JoinColumn(name = "visitor_id", nullable = false)

    private Visitor visitor;



    @ManyToOne(fetch = FetchType.LAZY)

    @JoinColumn(name = "exhibit_id", nullable = false)

    private Exhibit exhibit;



    @PrePersist

    protected void prePersist() { this.feedbackDate = LocalDateTime.now(); }



    public Long getFeedbackId() { return feedbackId; }

    public void setFeedbackId(Long feedbackId) { this.feedbackId = feedbackId; }

    public int getRating() { return rating; }
```

```java
    public void setRating(int rating) { this.rating = rating; }

    public String getComments() { return comments; }

    public void setComments(String comments) { this.comments = comments; }

    public LocalDateTime getFeedbackDate() { return feedbackDate; }

    public void setFeedbackDate(LocalDateTime feedbackDate) { this.feedbackDate = feedbackDate; }

    public Visitor getVisitor() { return visitor; }

    public void setVisitor(Visitor visitor) { this.visitor = visitor; }

    public Exhibit getExhibit() { return exhibit; }

    public void setExhibit(Exhibit exhibit) { this.exhibit = exhibit; }

}
```

7.2 Controller

- FeedbackController.java

```java
package com.museum.controller;



import com.museum.dto.FeedbackDTO;

import com.museum.model.Feedback;

import com.museum.service.FeedbackService;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;



import java.util.List;

import java.util.Map;



@RestController

@RequestMapping("/api/feedback")

public class FeedbackController {



    private final FeedbackService feedbackService;
```

```java
public FeedbackController(FeedbackService feedbackService) {
    this.feedbackService = feedbackService;
}

@PostMapping({"", "/"})
public ResponseEntity<?> submitFeedback(@RequestBody FeedbackDTO dto) {
    try {
        Feedback feedback = feedbackService.submitFeedback(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(Map.of(
                "message", "Feedback submitted. Thank you!",
                "feedbackId", feedback.getFeedbackId(),
                "rating", feedback.getRating()
        ));
    } catch (RuntimeException e) {
        return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
    }
}

@GetMapping({"", "/"})
public ResponseEntity<List<Feedback>> getAllFeedback() {
    return ResponseEntity.ok(feedbackService.getAllFeedback());
}

@GetMapping("/exhibit/{exhibitId}")
public ResponseEntity<List<Feedback>> getFeedbackByExhibit(@PathVariable Long exhibitId) {
    return ResponseEntity.ok(feedbackService.getFeedbackByExhibit(exhibitId));
}

@GetMapping("/visitor/{visitorId}")
public ResponseEntity<?> getFeedbackByVisitor(@PathVariable Long visitorId) {
```

```java
        List<Feedback> list = feedbackService.getFeedbackByVisitor(visitorId);

        // Return flat maps for frontend compatibility

        List<Map<String, Object>> result = list.stream().map(f -> {

            Map<String, Object> m = new java.util.LinkedHashMap<>();

            m.put("feedbackId",   f.getFeedbackId());

            m.put("rating",       f.getRating());

            m.put("comments",     f.getComments());

            m.put("feedbackDate", f.getFeedbackDate() != null ? f.getFeedbackDate().toString() : "");

            m.put("exhibitName",  f.getExhibit() != null ? f.getExhibit().getName() : "N/A");

            m.put("exhibitId",    f.getExhibit() != null ? f.getExhibit().getExhibitId() : 0);

            return m;

        }).toList();

        return ResponseEntity.ok(result);

    }



    @GetMapping("/exhibit/{exhibitId}/rating")

    public ResponseEntity<Map<String, Object>> getAverageRating(@PathVariable Long exhibitId) {

        double avg = feedbackService.getAverageRatingForExhibit(exhibitId);

        return ResponseEntity.ok(Map.of("exhibitId", exhibitId, "averageRating", avg));

    }

}
```

7.3 Service

- FeedbackService.java

```java
package com.museum.service;



import com.museum.dto.FeedbackDTO;

import com.museum.model.*;

import com.museum.repository.*;
```

```java
import org.springframework.stereotype.Service;

import org.springframework.transaction.annotation.Transactional;

import java.util.List;



@Service

public class FeedbackService {

    private final FeedbackRepository feedbackRepository;

    private final VisitorRepository visitorRepository;

    private final ExhibitRepository exhibitRepository;



    public FeedbackService(FeedbackRepository feedbackRepository,

                           VisitorRepository visitorRepository,

                           ExhibitRepository exhibitRepository) {

        this.feedbackRepository = feedbackRepository;

        this.visitorRepository = visitorRepository;

        this.exhibitRepository = exhibitRepository;

    }



    @Transactional

    public Feedback submitFeedback(FeedbackDTO dto) {

        Visitor visitor = visitorRepository.findById(dto.getVisitorId())

                .orElseThrow(() -> new RuntimeException("Visitor not found: " + dto.getVisitorId()));

        Exhibit exhibit = exhibitRepository.findById(dto.getExhibitId())

                .orElseThrow(() -> new RuntimeException("Exhibit not found: " + dto.getExhibitId()));

        Feedback f = new Feedback();

        f.setVisitor(visitor); f.setExhibit(exhibit);

        f.setRating(dto.getRating()); f.setComments(dto.getComments());

        return feedbackRepository.save(f);

    }
```

```
    @Transactional(readOnly = true) public List<Feedback> getAllFeedback() { return
feedbackRepository.findAll(); }

    @Transactional(readOnly = true) public List<Feedback> getFeedbackByExhibit(Long exhibitId) { return
feedbackRepository.findByExhibit_ExhibitId(exhibitId); }

    @Transactional(readOnly = true) public List<Feedback> getFeedbackByVisitor(Long visitorId) { return
feedbackRepository.findByVisitor_UserId(visitorId); }

    @Transactional(readOnly = true) public double getAverageRatingForExhibit(Long exhibitId) {

        Double avg = feedbackRepository.averageRatingByExhibit(exhibitId); return avg != null ? avg : 0.0;

    }

}
```

7.4 Repository

- FeedbackRepository.java

```
package com.museum.repository;



import com.museum.model.Feedback;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.jpa.repository.Query;

import org.springframework.data.repository.query.Param;

import org.springframework.stereotype.Repository;



import java.util.List;



@Repository

public interface FeedbackRepository extends JpaRepository<Feedback, Long> {



    List<Feedback> findByVisitor_UserId(Long visitorId);



    List<Feedback> findByExhibit_ExhibitId(Long exhibitId);



    @Query("SELECT AVG(f.rating) FROM Feedback f WHERE f.exhibit.exhibitId = :exhibitId")
```

```
    Double averageRatingByExhibit(@Param("exhibitId") Long exhibitId);

}
```

7.5 DTO

- FeedbackDTO.java

```
package com.museum.dto;



public class FeedbackDTO {

    private Long visitorId;

    private Long exhibitId;

    private int rating;

    private String comments;



    public Long getVisitorId() { return visitorId; }

    public void setVisitorId(Long v) { this.visitorId = v; }

    public Long getExhibitId() { return exhibitId; }

    public void setExhibitId(Long v) { this.exhibitId = v; }

    public int getRating() { return rating; }

    public void setRating(int v) { this.rating = v; }

    public String getComments() { return comments; }

    public void setComments(String v) { this.comments = v; }

}
```

8. Notification Module

8.1 Model

- Notification.java

```
    package com.museum.model;



    import jakarta.persistence.*;
```

```java
import java.time.LocalDateTime;



@Entity

@Table(name = "notifications")

public class Notification {



    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long notificationId;



    @Column(nullable = false, columnDefinition = "TEXT")

    private String message;



    @Column(nullable = false, updatable = false)

    private LocalDateTime createdOn;



    @Enumerated(EnumType.STRING)

    @Column(nullable = false)

    private NotificationStatus status = NotificationStatus.SENT;



    @ManyToOne(fetch = FetchType.LAZY)

    @JoinColumn(name = "user_id", nullable = false)

    private User user;



    @PrePersist

    protected void prePersist() { this.createdOn = LocalDateTime.now(); }



    public Long getNotificationId() { return notificationId; }

    public void setNotificationId(Long notificationId) { this.notificationId = notificationId; }

    public String getMessage() { return message; }

    public void setMessage(String message) { this.message = message; }
```

```
        public LocalDateTime getCreatedOn() { return createdOn; }

        public void setCreatedOn(LocalDateTime createdOn) { this.createdOn = createdOn; }

        public NotificationStatus getStatus() { return status; }

        public void setStatus(NotificationStatus status) { this.status = status; }

        public User getUser() { return user; }

        public void setUser(User user) { this.user = user; }

    }
```

- NotificationStatus.java

```
package com.museum.model;



public enum NotificationStatus {

    SENT,

    READ

}
```

8.2 Controller

- NotificationController.java

```
package com.museum.controller;



import com.museum.model.Notification;

import com.museum.service.NotificationManager;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;



import java.util.List;

import java.util.Map;



@RestController

@RequestMapping("/api/notifications")

public class NotificationController {
```

```java
    private final NotificationManager notificationManager;


    public NotificationController(NotificationManager notificationManager) {
        this.notificationManager = notificationManager;
    }


    @GetMapping("/user/{userId}")
    public ResponseEntity<List<Notification>> getForUser(@PathVariable Long userId) {
        return ResponseEntity.ok(notificationManager.getNotificationsForUser(userId));
    }


    @GetMapping("/user/{userId}/unread")
    public ResponseEntity<List<Notification>> getUnread(@PathVariable Long userId) {
        return ResponseEntity.ok(notificationManager.getUnreadNotifications(userId));
    }


    @GetMapping("/user/{userId}/count")
    public ResponseEntity<Map<String, Object>> getCount(@PathVariable Long userId) {
        return ResponseEntity.ok(Map.of("userId", userId, "unreadCount",
notificationManager.countUnread(userId)));
    }


    @PutMapping("/{id}/read")
    public ResponseEntity<?> markRead(@PathVariable Long id) {
        try {
            Notification n = notificationManager.markAsRead(id);
            return ResponseEntity.ok(Map.of("notificationId", n.getNotificationId(), "status",
n.getStatus()));
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
        }
```

```
    }


    @PutMapping("/user/{userId}/read-all")
    public ResponseEntity<Map<String, String>> markAllRead(@PathVariable Long userId) {
        notificationManager.markAllAsRead(userId);
        return ResponseEntity.ok(Map.of("message", "All notifications marked as read."));
    }
}
```

8.3 Service

- NotificationManager.java

```
package com.museum.service;


import com.museum.model.Notification;
import com.museum.model.NotificationStatus;
import com.museum.model.User;
import com.museum.repository.NotificationRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;


@Service
public class NotificationManager {


    private static NotificationManager instance;
    private final NotificationRepository notificationRepository;


    public NotificationManager(NotificationRepository notificationRepository) {
        this.notificationRepository = notificationRepository;
        if (instance == null) instance = this;
    }
```

```java
public static NotificationManager getInstance() { return instance; }


@Transactional
public Notification notifyUser(User user, String message) {
    Notification n = new Notification();
    n.setUser(user); n.setMessage(message); n.setStatus(NotificationStatus.SENT);
    return notificationRepository.save(n);
}


@Transactional(readOnly = true)
public List<Notification> getNotificationsForUser(Long userId) {
    return notificationRepository.findByUser_UserIdOrderByCreatedOnDesc(userId);
}


@Transactional(readOnly = true)
public List<Notification> getUnreadNotifications(Long userId) {
    return notificationRepository.findByUser_UserIdAndStatus(userId, NotificationStatus.SENT);
}


@Transactional(readOnly = true)
public long countUnread(Long userId) {
    return notificationRepository.countByUser_UserIdAndStatus(userId, NotificationStatus.SENT);
}


@Transactional
public Notification markAsRead(Long notificationId) {
    Notification n = notificationRepository.findById(notificationId)
            .orElseThrow(() -> new RuntimeException("Notification not found: " + notificationId));
    n.setStatus(NotificationStatus.READ);
    return notificationRepository.save(n);
```

```
    }


    @Transactional

    public void markAllAsRead(Long userId) {

        List<Notification> unread = getUnreadNotifications(userId);

        unread.forEach(n -> n.setStatus(NotificationStatus.READ));

        notificationRepository.saveAll(unread);

    }

}
```

8.4 Repository

- NotificationRepository.java

```
package com.museum.repository;


import com.museum.model.Notification;

import com.museum.model.NotificationStatus;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;


import java.util.List;


@Repository

public interface NotificationRepository extends JpaRepository<Notification, Long> {


    List<Notification> findByUser_UserIdOrderByCreatedOnDesc(Long userId);


    List<Notification> findByUser_UserIdAndStatus(Long userId, NotificationStatus status);


    long countByUser_UserIdAndStatus(Long userId, NotificationStatus status);

}
```

9. Report & Analytics Module

9.1 Model

- Report.java

```
package com.museum.model;


import jakarta.persistence.*;

import java.time.LocalDateTime;


@Entity

@Table(name = "reports")

public class Report {


    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long reportId;


    @Column(nullable = false)

    private String reportType;


    @Column(nullable = false, updatable = false)

    private LocalDateTime generatedOn;


    @Column(columnDefinition = "TEXT")

    private String reportData;


    @ManyToOne(fetch = FetchType.LAZY)

    @JoinColumn(name = "generated_by")

    private Admin generatedBy;


    @PrePersist
```

```
    protected void prePersist() { this.generatedOn = LocalDateTime.now(); }



    public Long getReportId() { return reportId; }

    public void setReportId(Long reportId) { this.reportId = reportId; }

    public String getReportType() { return reportType; }

    public void setReportType(String reportType) { this.reportType = reportType; }

    public LocalDateTime getGeneratedOn() { return generatedOn; }

    public void setGeneratedOn(LocalDateTime generatedOn) { this.generatedOn = generatedOn; }

    public String getReportData() { return reportData; }

    public void setReportData(String reportData) { this.reportData = reportData; }

    public Admin getGeneratedBy() { return generatedBy; }

    public void setGeneratedBy(Admin generatedBy) { this.generatedBy = generatedBy; }

}
```

9.2 Controller

- AdminController.java

```
package com.museum.controller;



import com.museum.dto.AnalyticsDTO;

import com.museum.model.*;

import com.museum.service.*;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;



import java.util.List;

import java.util.Map;



@RestController

@RequestMapping("/api/admin")

public class AdminController {
```

```java
private final UserService userService;

private final GuideAssignmentService assignmentService;

private final BookingService bookingService;

private final ReportService reportService;

private final AnalyticsService analyticsService;

private final ExhibitService exhibitService;


public AdminController(UserService userService,

                       GuideAssignmentService assignmentService,

                       BookingService bookingService,

                       ReportService reportService,

                       AnalyticsService analyticsService,

                       ExhibitService exhibitService) {

    this.userService = userService;

    this.assignmentService = assignmentService;

    this.bookingService = bookingService;

    this.reportService = reportService;

    this.analyticsService = analyticsService;

    this.exhibitService = exhibitService;

}


@GetMapping("/analytics")

public ResponseEntity<AnalyticsDTO> getAnalytics() {

    return ResponseEntity.ok(analyticsService.getDashboardAnalytics());

}


@GetMapping("/users")

public ResponseEntity<List<User>> getAllUsers() {

    return ResponseEntity.ok(userService.getAllUsers());

}
```

```java
@PostMapping("/exhibits")
public ResponseEntity<?> addExhibit(@RequestBody Exhibit exhibit) {
    try {
        return ResponseEntity.status(HttpStatus.CREATED).body(exhibitService.addExhibit(exhibit));
    } catch (RuntimeException e) {
        return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
    }
}

@PutMapping("/exhibits/{id}")
public ResponseEntity<?> updateExhibit(@PathVariable Long id, @RequestBody Exhibit exhibit) {
    try {
        return ResponseEntity.ok(exhibitService.updateExhibit(id, exhibit));
    } catch (RuntimeException e) {
        return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
    }
}

@DeleteMapping("/exhibits/{id}")
public ResponseEntity<?> deleteExhibit(@PathVariable Long id) {
    try {
        exhibitService.deleteExhibit(id);
        return ResponseEntity.ok(Map.of("message", "Exhibit archived successfully."));
    } catch (RuntimeException e) {
        return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
    }
}

@GetMapping("/exhibits")
public ResponseEntity<?> getAllExhibits() {
```

```
        return ResponseEntity.ok(exhibitService.getAllExhibits());
    }

    @GetMapping({"/bookings", "/bookings/"})
    public ResponseEntity<?> getAllBookings() {
        return ResponseEntity.ok(bookingService.getAllBookingsFlat());
    }

    @GetMapping("/bookings/pending")
    public ResponseEntity<?> getPendingBookings() {
        return ResponseEntity.ok(bookingService.getPendingBookingsFlat());
    }

    @PutMapping("/bookings/{id}/confirm")
    public ResponseEntity<?> confirmBooking(@PathVariable Long id) {
        try {
            return ResponseEntity.ok(bookingService.confirmBooking(id));
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
        }
    }

    @PutMapping("/bookings/{id}/reject")
    public ResponseEntity<?> rejectBooking(@PathVariable Long id) {
        try {
            return ResponseEntity.ok(bookingService.rejectBooking(id));
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
        }
    }
```

```java
@GetMapping({"/schedules", "/schedules/"})
public ResponseEntity<?> getAllSchedules() { return getAllBookings(); }


@GetMapping("/schedules/pending")
public ResponseEntity<?> getPendingSchedules() { return getPendingBookings(); }


@PutMapping("/schedules/{id}/approve")
public ResponseEntity<?> approveSchedule(@PathVariable Long id) { return confirmBooking(id); }


@PutMapping("/schedules/{id}/reject")
public ResponseEntity<?> rejectSchedule(@PathVariable Long id) { return rejectBooking(id); }


@PostMapping("/guides/create")
public ResponseEntity<?> createGuide(@RequestBody Map<String, Object> body) {
    try {
        String name = (String) body.get("name");
        String email = (String) body.get("email");
        String password = (String) body.get("password");
        String specialization = (String) body.getOrDefault("specialization", "General");
        if (name == null || email == null || password == null)
            return ResponseEntity.badRequest().body(Map.of("error", "name, email, password required"));


        com.museum.dto.RegisterDTO dto = new com.museum.dto.RegisterDTO();
        dto.setName(name); dto.setEmail(email); dto.setPassword(password);
        dto.setRole(UserRole.GUIDE); dto.setSpecialization(specialization);
        User guide = userService.register(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(Map.of(
                "message", "Guide created", "userId", guide.getUserId(),
                "id", guide.getUserId(), "name", guide.getName(),
                "email", guide.getEmail(), "role", guide.getRole()
        ));
```

```java
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
        }
    }

    @GetMapping({"/guides", "/guides/"})
    public ResponseEntity<?> getAllGuides() {
        List<Guide> guides = assignmentService.getAllGuides();
        List<Map<String, Object>> result = guides.stream().map(g -> Map.<String, Object>of(
                "userId", g.getUserId(), "id", g.getUserId(), "name", g.getName(),
                "email", g.getEmail(),
                "specialization", g.getSpecialization() != null ? g.getSpecialization() : "",
                "available", g.isAvailable()
        )).toList();
        return ResponseEntity.ok(result);
    }

    @PostMapping("/guides/assign")
    public ResponseEntity<?> assignGuide(@RequestBody Map<String, Object> body) {
        try {
            Long guideId = Long.valueOf(body.get("guideId").toString());
            Long bookingId = Long.valueOf(body.get("bookingId").toString());
            Map<String, Object> result = assignmentService.assignGuideToBooking(guideId, bookingId);
            return ResponseEntity.status(HttpStatus.CREATED).body(result);
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
        }
    }

    @DeleteMapping("/guides/{id}")
    public ResponseEntity<?> deleteGuide(@PathVariable Long id) {
```

```
        try {
            userService.deleteGuide(id);
            return ResponseEntity.ok(Map.of("message", "Guide deleted successfully."));
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
        }
    }

    @GetMapping("/guides/assignments")
    public ResponseEntity<?> getAllAssignments() {
        return ResponseEntity.ok(assignmentService.getAllAssignmentsFlat());
    }

    @PostMapping("/reports/generate")
    public ResponseEntity<?> generateReport(@RequestBody Map<String, Object> body) {
        try {
            String reportType = (String) body.get("reportType");
            Long adminId = Long.valueOf(body.get("adminId").toString());
            Map<String, Object> result = reportService.generateReport(reportType, adminId);
            return ResponseEntity.status(HttpStatus.CREATED).body(result);
        } catch (RuntimeException e) {
            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
        }
    }

    @GetMapping("/reports")
    public ResponseEntity<?> getAllReports() {
        return ResponseEntity.ok(reportService.getAllReportsFlat());
    }

    @GetMapping("/reports/type/{type}")
```

```
    public ResponseEntity<?> getReportsByType(@PathVariable String type) {

        return ResponseEntity.ok(reportService.getReportsByTypeFlat(type));

    }



    @DeleteMapping("/reports/{id}")

    public ResponseEntity<?> deleteReport(@PathVariable Long id) {

        try {

            reportService.deleteReport(id);

            return ResponseEntity.ok(Map.of("message", "Report deleted successfully."));

        } catch (RuntimeException e) {

            return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));

        }

    }

}
```

9.3 Service

- ReportService.java

```
package com.museum.service;



import com.museum.model.Report;

import com.museum.model.Ticket;

import com.museum.model.Exhibit;

import com.museum.repository.AdminRepository;

import com.museum.repository.ReportRepository;

import com.museum.repository.TicketRepository;

import com.museum.repository.ExhibitRepository;

import com.museum.repository.BookingRepository;

import org.springframework.stereotype.Service;

import org.springframework.transaction.annotation.Transactional;



import java.util.LinkedHashMap;
```

```java
import java.util.List;

import java.util.Map;

import java.util.stream.Collectors;



@Service

public class ReportService {



    private final ReportRepository reportRepository;

    private final AdminRepository adminRepository;

    private final AnalyticsService analyticsService;

    private final TicketRepository ticketRepository;

    private final ExhibitRepository exhibitRepository;

    private final BookingRepository bookingRepository;



    public ReportService(ReportRepository reportRepository, AdminRepository adminRepository,

                         AnalyticsService analyticsService, TicketRepository ticketRepository,

                         ExhibitRepository exhibitRepository, BookingRepository bookingRepository) {

        this.reportRepository = reportRepository;

        this.adminRepository = adminRepository;

        this.analyticsService = analyticsService;

        this.ticketRepository = ticketRepository;

        this.exhibitRepository = exhibitRepository;

        this.bookingRepository = bookingRepository;

    }



    @Transactional

    public Map<String, Object> generateReport(String reportType, Long adminId) {

        com.museum.model.Admin admin = adminRepository.findById(adminId)

                .orElseThrow(() -> new RuntimeException("Admin not found: " + adminId));



        String summary;
```

```
        Map<String, Object> details = new LinkedHashMap<>();



        switch (reportType.toUpperCase()) {

            case "TICKET" -> {

                List<Ticket> tickets = ticketRepository.findAll();

                long paid = tickets.stream().filter(t -> "PAID".equals(t.getPaymentStatus())).count();

                long pending = tickets.stream().filter(t ->
"PENDING".equals(t.getPaymentStatus())).count();

                long refunded = tickets.stream().filter(t ->
"REFUNDED".equals(t.getPaymentStatus())).count();

                double revenue = tickets.stream().filter(t -> "PAID".equals(t.getPaymentStatus()))

                        .mapToDouble(Ticket::getPrice).sum();

                summary = "Total Tickets: " + tickets.size() + " | Paid: " + paid +

                        " | Pending: " + pending + " | Refunded: " + refunded +

                        " | Revenue: ₹" + String.format("%.2f", revenue);

                details.put("totalTickets", tickets.size());

                details.put("paid", paid);

                details.put("pending", pending);

                details.put("refunded", refunded);

                details.put("totalRevenue", revenue);

                Map<String, Long> byType = tickets.stream()

                        .collect(Collectors.groupingBy(t -> t.getTicketType() != null ?
t.getTicketType().name() : "UNKNOWN", Collectors.counting()));

                details.put("byType", byType);

            }

            case "EXHIBIT" -> {

                List<Exhibit> exhibits = exhibitRepository.findAll();

                long active = exhibits.stream().filter(e -> "ACTIVE".equals(e.getStatus())).count();

                long archived = exhibits.stream().filter(e -> "ARCHIVED".equals(e.getStatus())).count();

                summary = "Total Exhibits: " + exhibits.size() + " | Active: " + active + " | Archived: " +
archived;

                details.put("totalExhibits", exhibits.size());

                details.put("active", active);
```

```
            details.put("archived", archived);

            Map<String, Long> byCategory = exhibits.stream()

                    .filter(e -> e.getCategory() != null)

                    .collect(Collectors.groupingBy(Exhibit::getCategory, Collectors.counting()));

            details.put("byCategory", byCategory);

            List<Map<String, Object>> exhibitList = exhibits.stream().map(e -> {

                Map<String, Object> em = new LinkedHashMap<>();

                em.put("name", e.getName());

                em.put("category", e.getCategory());

                em.put("status", e.getStatus());

                return em;

            }).toList();

            details.put("exhibits", exhibitList);

        }

        case "ANALYTICS" -> {

            var a = analyticsService.getDashboardAnalytics();

            summary = "Visitors: " + a.getTotalVisitors() + " | Bookings: " + a.getTotalBookings()

                    + " | Revenue: ₹" + String.format("%.2f", a.getTotalRevenue())

                    + " | Avg Rating: " + String.format("%.1f", a.getAverageFeedbackRating());

            details.put("totalVisitors", a.getTotalVisitors());

            details.put("totalExhibits", a.getTotalExhibits());

            details.put("totalBookings", a.getTotalBookings());

            details.put("totalTickets", a.getTotalTickets());

            details.put("totalRevenue", a.getTotalRevenue());

            details.put("pendingBookings", a.getPendingSchedules());

            details.put("activeExhibits", a.getActiveExhibits());

            details.put("avgRating", a.getAverageFeedbackRating());

            details.put("bookingsByStatus", a.getBookingsByStatus());

            details.put("ticketsByType", a.getTicketsByType());

            details.put("exhibitsByCategory", a.getExhibitsByCategory());

        }
```

```
            case "COMBINED" -> {

                List<Exhibit> exhibits = exhibitRepository.findAll();

                List<Ticket> tickets = ticketRepository.findAll();

                var allBookings = bookingRepository.findAll();

                double revenue = tickets.stream()

                        .filter(t -> "PAID".equals(t.getPaymentStatus()))

                        .mapToDouble(Ticket::getPrice).sum();

                long totalBookings = allBookings.size();



                List<Map<String, Object>> exhibitBreakdown = exhibits.stream().map(ex -> {

                    long exBookings = allBookings.stream()

                            .filter(b -> b.getSchedule() != null

                                    && b.getSchedule().getExhibit() != null

                                    &&
ex.getExhibitId().equals(b.getSchedule().getExhibit().getExhibitId()))

                            .count();

                    double exRevenue = tickets.stream()

                            .filter(t -> "PAID".equals(t.getPaymentStatus())

                                    && t.getBooking() != null

                                    && t.getBooking().getSchedule() != null

                                    && t.getBooking().getSchedule().getExhibit() != null

                                    &&
ex.getExhibitId().equals(t.getBooking().getSchedule().getExhibit().getExhibitId()))

                            .mapToDouble(Ticket::getPrice).sum();

                    long assignedGuides = allBookings.stream()

                            .filter(b -> b.getSchedule() != null

                                    && b.getSchedule().getExhibit() != null

                                    &&
ex.getExhibitId().equals(b.getSchedule().getExhibit().getExhibitId()))

                            .flatMap(b -> b.getGuideAssignments().stream())

                            .filter(a -> "CONFIRMED".equals(a.getStatus()))

                            .map(a -> a.getGuide().getUserId())
```

```
                        .distinct().count();

                Map<String, Object> em = new LinkedHashMap<>();

                em.put("exhibitName", ex.getName());

                em.put("category", ex.getCategory() != null ? ex.getCategory() : "");

                em.put("status", ex.getStatus());

                em.put("totalBookings", exBookings);

                em.put("revenue", exRevenue);

                em.put("assignedGuides", assignedGuides);

                return em;

            }).toList();



            summary = "Exhibits: " + exhibits.size() + " | Total Bookings: " + totalBookings

                    + " | Total Revenue: ₹" + String.format("%.2f", revenue);

            details.put("totalExhibits", exhibits.size());

            details.put("totalBookings", totalBookings);

            details.put("totalTickets", tickets.size());

            details.put("totalRevenue", revenue);

            details.put("exhibitBreakdown", exhibitBreakdown);

        }

        default -> throw new RuntimeException("Unknown report type: " + reportType +

                ". Valid types: TICKET, EXHIBIT, ANALYTICS, COMBINED");

    }



    String dataJson = mapToJson(details);



    Report r = new Report();

    r.setReportType(reportType.toUpperCase());

    r.setReportData(dataJson);

    r.setGeneratedBy(admin);

    r = reportRepository.save(r);
```

```java
        Map<String, Object> response = new LinkedHashMap<>();

        response.put("reportId", r.getReportId());

        response.put("reportType", r.getReportType());

        response.put("generatedOn", r.getGeneratedOn() != null ? r.getGeneratedOn().toString() : "");

        response.put("summary", summary);

        response.put("details", details);

        return response;

    }



    @Transactional(readOnly = true)

    public List<Map<String, Object>> getAllReportsFlat() {

        return reportRepository.findAll().stream().map(r -> {

            Map<String, Object> m = new LinkedHashMap<>();

            m.put("reportId", r.getReportId());

            m.put("reportType", r.getReportType());

            m.put("generatedOn", r.getGeneratedOn() != null ? r.getGeneratedOn().toString() : "");

            m.put("reportData", r.getReportData() != null ? r.getReportData() : "{}");

            return m;

        }).toList();

    }



    @Transactional(readOnly = true)

    public List<Map<String, Object>> getReportsByTypeFlat(String type) {

        return reportRepository.findByReportType(type.toUpperCase()).stream().map(r -> {

            Map<String, Object> m = new LinkedHashMap<>();

            m.put("reportId", r.getReportId());

            m.put("reportType", r.getReportType());

            m.put("generatedOn", r.getGeneratedOn() != null ? r.getGeneratedOn().toString() : "");

            m.put("reportData", r.getReportData() != null ? r.getReportData() : "{}");

            return m;

        }).toList();
```

```java
}


@Transactional
public void deleteReport(Long reportId) {
    if (!reportRepository.existsById(reportId))
        throw new RuntimeException("Report not found: " + reportId);
    reportRepository.deleteById(reportId);
}


@SuppressWarnings("unchecked")
private String mapToJson(Map<String, Object> map) {
    StringBuilder sb = new StringBuilder("{");
    boolean first = true;
    for (Map.Entry<String, Object> e : map.entrySet()) {
        if (!first) sb.append(",");
        first = false;
        sb.append("\"").append(e.getKey()).append("\":");
        Object v = e.getValue();
        if (v == null) sb.append("null");
        else if (v instanceof Number) sb.append(v);
        else if (v instanceof Boolean) sb.append(v);
        else if (v instanceof Map) sb.append(mapToJson((Map<String, Object>) v));
        else if (v instanceof List) sb.append(listToJson((List<?>) v));
        else sb.append("\"").append(v.toString().replace("\"", "\\\"")).append("\"");
    }
    sb.append("}");
    return sb.toString();
}


@SuppressWarnings("unchecked")
private String listToJson(List<?> list) {
```

```
        StringBuilder sb = new StringBuilder("[");

        boolean first = true;

        for (Object item : list) {

            if (!first) sb.append(",");

            first = false;

            if (item instanceof Map) sb.append(mapToJson((Map<String, Object>) item));

            else if (item instanceof Number || item instanceof Boolean) sb.append(item);

            else sb.append("\"").append(item.toString().replace("\"", "\\\"")).append("\"");

        }

        return sb.append("]").toString();

    }

}
```

- AnalyticsService.java

```
package com.museum.service;



import com.museum.dto.AnalyticsDTO;

import com.museum.model.*;

import com.museum.repository.*;

import org.springframework.stereotype.Service;

import org.springframework.transaction.annotation.Transactional;

import java.util.List;

import java.util.Map;

import java.util.stream.Collectors;



@Service

public class AnalyticsService {



    private final VisitorRepository visitorRepository;

    private final ExhibitRepository exhibitRepository;

    private final BookingRepository bookingRepository;
```

```java
    private final TicketRepository ticketRepository;

    private final FeedbackRepository feedbackRepository;

    private final ScheduleRepository scheduleRepository;

    private final GuideRepository guideRepository;



    public AnalyticsService(VisitorRepository visitorRepository, ExhibitRepository exhibitRepository,

                            BookingRepository bookingRepository, TicketRepository ticketRepository,

                            FeedbackRepository feedbackRepository, ScheduleRepository scheduleRepository,

                            GuideRepository guideRepository) {

        this.visitorRepository = visitorRepository; this.exhibitRepository = exhibitRepository;

        this.bookingRepository = bookingRepository; this.ticketRepository = ticketRepository;

        this.feedbackRepository = feedbackRepository; this.scheduleRepository = scheduleRepository;

        this.guideRepository = guideRepository;

    }



    @Transactional(readOnly = true)

    public AnalyticsDTO getDashboardAnalytics() {

        long totalVisitors  = visitorRepository.count();

        long totalExhibits  = exhibitRepository.count();

        long totalBookings  = bookingRepository.count();

        long totalTickets   = ticketRepository.count();

        double totalRevenue = ticketRepository.totalRevenue() != null ? ticketRepository.totalRevenue() :
0.0;

        long pendingSchedules = scheduleRepository.findByStatus(ScheduleStatus.PENDING).size();

        long activeExhibits   = exhibitRepository.findByStatus("ACTIVE").size();

        long availableGuides  = guideRepository.findByAvailable(true).size();



        double avgRating = feedbackRepository.findAll().stream()

                .mapToInt(Feedback::getRating).average().orElse(0.0);



        Map<String,Long> bookingsByStatus = bookingRepository.findAll().stream()
```

```
                .collect(Collectors.groupingBy(Booking::getStatus, Collectors.counting()));



        Map<String,Long> ticketsByType = ticketRepository.findAll().stream()

                .collect(Collectors.groupingBy(t -> t.getTicketType().name(), Collectors.counting()));



        Map<String,Double> revenueByType = ticketRepository.findAll().stream()

                .collect(Collectors.groupingBy(t -> t.getTicketType().name(),
Collectors.summingDouble(Ticket::getPrice)));



        Map<String,Long> exhibitsByCategory = exhibitRepository.findAll().stream()

                .filter(e -> e.getCategory() != null)

                .collect(Collectors.groupingBy(Exhibit::getCategory, Collectors.counting()));



        return new AnalyticsDTO(totalVisitors, totalExhibits, totalBookings, totalTickets, totalRevenue,

                pendingSchedules, activeExhibits, availableGuides, avgRating,

                bookingsByStatus, ticketsByType, revenueByType, exhibitsByCategory);

    }



    @Transactional(readOnly = true) public List<Ticket> getTicketReport() { return
ticketRepository.findAll(); }

    @Transactional(readOnly = true) public List<Exhibit> getExhibitReport() { return
exhibitRepository.findAll(); }

}
```

9.4 Repository

- ReportRepository.java

```
package com.museum.repository;



import com.museum.model.Report;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;
```

```java
import java.util.List;


@Repository
public interface ReportRepository extends JpaRepository<Report, Long> {


    List<Report> findByReportType(String reportType);


    List<Report> findByGeneratedBy_UserId(Long adminId);
}
```

9.5 DTO

- AnalyticsDTO.java

```java
package com.museum.dto;


import java.util.Map;


public class AnalyticsDTO {


    private long totalVisitors;
    private long totalExhibits;
    private long totalBookings;
    private long totalTickets;
    private double totalRevenue;
    private long pendingSchedules;
    private long activeExhibits;
    private long availableGuides;
    private double averageFeedbackRating;
    private Map<String, Long> bookingsByStatus;
    private Map<String, Long> ticketsByType;
```

```java
    private Map<String, Double> revenueByTicketType;

    private Map<String, Long> exhibitsByCategory;



    public AnalyticsDTO(long totalVisitors, long totalExhibits, long totalBookings,

                        long totalTickets, double totalRevenue, long pendingSchedules,

                        long activeExhibits, long availableGuides, double averageFeedbackRating,

                        Map<String, Long> bookingsByStatus, Map<String, Long> ticketsByType,

                        Map<String, Double> revenueByTicketType, Map<String, Long> exhibitsByCategory) {

        this.totalVisitors = totalVisitors;

        this.totalExhibits = totalExhibits;

        this.totalBookings = totalBookings;

        this.totalTickets = totalTickets;

        this.totalRevenue = totalRevenue;

        this.pendingSchedules = pendingSchedules;

        this.activeExhibits = activeExhibits;

        this.availableGuides = availableGuides;

        this.averageFeedbackRating = averageFeedbackRating;

        this.bookingsByStatus = bookingsByStatus;

        this.ticketsByType = ticketsByType;

        this.revenueByTicketType = revenueByTicketType;

        this.exhibitsByCategory = exhibitsByCategory;

    }



    public long getTotalVisitors() { return totalVisitors; }

    public long getTotalExhibits() { return totalExhibits; }

    public long getTotalBookings() { return totalBookings; }

    public long getTotalTickets() { return totalTickets; }

    public double getTotalRevenue() { return totalRevenue; }

    public long getPendingSchedules() { return pendingSchedules; }

    public long getActiveExhibits() { return activeExhibits; }

    public long getAvailableGuides() { return availableGuides; }
```

```
    public double getAverageFeedbackRating() { return averageFeedbackRating; }

    public Map<String, Long> getBookingsByStatus() { return bookingsByStatus; }

    public Map<String, Long> getTicketsByType() { return ticketsByType; }

    public Map<String, Double> getRevenueByTicketType() { return revenueByTicketType; }

    public Map<String, Long> getExhibitsByCategory() { return exhibitsByCategory; }

}
```

10. View Layer (Frontend)

10.1 View

- index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Museum Management System</title>

<style>

*{margin:0;padding:0;box-sizing:border-box}

body{font-family:'Segoe UI',sans-serif;background:#f0f2f5;color:#333;min-height:100vh}



nav{background:linear-gradient(135deg,#4f46e5,#7c3aed);color:#fff;padding:14px
28px;display:flex;justify-content:space-between;align-items:center;box-shadow:0 2px 12px
rgba(0,0,0,.2);position:sticky;top:0;z-index:200}

nav h1{font-size:20px;letter-spacing:.3px}

.nav-right{display:flex;align-items:center;gap:12px;font-size:14px}



.page{display:none}.page.active{display:block}

.container{max-width:1200px;margin:0 auto;padding:24px}



button{padding:9px
18px;cursor:pointer;border:none;border-radius:7px;font-size:14px;font-weight:500;transition:all
.18s;font-family:inherit}
```

```
.btn{background:#4f46e5;color:#fff}

.btn:hover{background:#4338ca}

.btn-green{background:#16a34a;color:#fff}

.btn-green:hover{background:#15803d}

.btn-red{background:#dc2626;color:#fff}

.btn-red:hover{background:#b91c1c}

.btn-gray{background:#6b7280;color:#fff}

.btn-gray:hover{background:#4b5563}

.btn-outline{background:#fff;color:#4f46e5;border:2px solid #4f46e5}

.btn-outline:hover{background:#eef2ff}

.btn-sm{padding:5px 12px;font-size:13px}

.btn-danger{background:#dc2626;color:#fff;border:none}

.btn-danger:hover{background:#b91c1c}

button:disabled{opacity:.5;cursor:not-allowed;transform:none!important}



.form-group{margin-bottom:16px}

label{display:block;margin-bottom:6px;font-weight:500;font-size:13px;color:#555}

input,select,textarea{width:100%;padding:10px 12px;border:1.5px solid
#d1d5db;border-radius:7px;font-size:14px;font-family:inherit;transition:border-color .2s;background:#fff}

input:focus,select:focus,textarea:focus{outline:none;border-color:#4f46e5;box-shadow:0 0 0 3px
rgba(79,70,229,.12)}



.card{background:#fff;border-radius:12px;padding:22px;box-shadow:0 1px 8px
rgba(0,0,0,.08);transition:transform .2s,box-shadow .2s}

.card:hover{transform:translateY(-3px);box-shadow:0 4px 20px rgba(0,0,0,.12)}

.card h3{color:#4f46e5;margin-bottom:10px;font-size:16px}

.card p{color:#555;font-size:14px;margin:5px 0;line-height:1.5}

.grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(270px,1fr));gap:18px;margin-top:16px}

.form-card{background:#fff;padding:28px;border-radius:12px;box-shadow:0 2px 12px
rgba(0,0,0,.08);max-width:460px;margin:40px auto}



.stats{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:14px;margin-bottom:26px}
```

```
.stat{background:linear-gradient(135deg,#4f46e5,#7c3aed);color:#fff;padding:20px;border-radius:12px;text-al
ign:center}

.stat
h4{font-size:12px;color:rgba(255,255,255,.82);text-transform:uppercase;letter-spacing:.6px;margin-bottom:8p
x}

.stat .n{font-size:34px;font-weight:700}



.tabs{display:flex;gap:2px;margin-bottom:22px;border-bottom:2px solid
#e5e7eb;padding-bottom:0;flex-wrap:wrap}

.tab-btn{padding:10px
20px;border:none;background:none;cursor:pointer;font-size:14px;font-weight:500;color:#6b7280;border-bottom:
3px solid transparent;margin-bottom:-2px;transition:all .22s;border-radius:6px 6px 0 0}

.tab-btn.active{color:#4f46e5;border-bottom-color:#4f46e5;background:#eef2ff}

.tab-content{display:none}.tab-content.active{display:block}



table{width:100%;border-collapse:collapse;background:#fff;border-radius:10px;overflow:hidden;box-shadow:0
1px 8px rgba(0,0,0,.07)}

th,td{padding:12px 15px;text-align:left;border-bottom:1px solid #f1f5f9;font-size:13.5px}

th{background:#4f46e5;color:#fff;font-size:12px;text-transform:uppercase;letter-spacing:.5px;font-weight:60
0}

tr:last-child td{border:none}

tr:hover td{background:#fafbff}



.badge{display:inline-block;padding:3px 11px;border-radius:20px;font-size:11.5px;font-weight:600}

.b-pending{background:#fef9c3;color:#854d0e}

.b-confirmed{background:#dcfce7;color:#166534}

.b-assigned{background:#dbeafe;color:#1e40af}

.b-cancelled,.b-rejected,.b-refunded{background:#fee2e2;color:#991b1b}

.b-paid{background:#dcfce7;color:#166534}

.b-active{background:#dcfce7;color:#166534}

.b-archived{background:#f1f5f9;color:#475569}



.alert{padding:13px 16px;border-radius:8px;margin-bottom:16px;font-size:14px;line-height:1.5}

.alert-ok{background:#dcfce7;color:#166534;border:1px solid #bbf7d0}

.alert-err{background:#fee2e2;color:#991b1b;border:1px solid #fecaca}
```

```
.alert-info{background:#dbeafe;color:#1e40af;border:1px solid #bfdbfe}



.modal{display:none;position:fixed;inset:0;background:rgba(0,0,0,.5);z-index:500;align-items:center;justify
-content:center}

.modal.open{display:flex}

.modal-box{background:#fff;padding:30px;border-radius:14px;max-width:520px;width:92%;box-shadow:0 8px 32px
rgba(0,0,0,.18);position:relative;max-height:90vh;overflow-y:auto}

.modal-box h3{margin-bottom:18px;color:#1f2937;font-size:18px}

.modal-close{position:absolute;top:14px;right:16px;font-size:22px;cursor:pointer;color:#9ca3af;background:n
one;border:none;padding:0}

.modal-close:hover{color:#374151}



.notif{background:#eef2ff;border-left:4px solid #4f46e5;padding:13px 16px;border-radius:7px;margin:8px
0;font-size:14px}

.notif small{display:block;margin-top:4px;color:#6b7280;font-size:12px}



.sh{display:flex;justify-content:space-between;align-items:center;margin-bottom:16px;flex-wrap:wrap;gap:8px
}

.sh h3{color:#1f2937;font-size:17px;margin:0}



.ph{margin-bottom:22px;padding-bottom:14px;border-bottom:2px solid #4f46e5}

.ph h2{font-size:22px;color:#1f2937}



.empty{text-align:center;padding:40px;color:#9ca3af;font-size:15px}



.ticket{background:linear-gradient(135deg,#f8faff,#eef2ff);border:2px solid
#c7d2fe;border-radius:12px;padding:20px;position:relative;overflow:hidden}

.ticket h4{color:#4f46e5;font-size:16px;margin-bottom:12px}

.ticket .tk-row{display:flex;justify-content:space-between;margin:6px 0;font-size:13.5px}

.ticket .tk-row span:first-child{color:#6b7280}

.ticket .tk-row strong{color:#1f2937}



.report-section{background:#fff;border-radius:10px;padding:20px;box-shadow:0 1px 8px
rgba(0,0,0,.07);margin-bottom:14px}
```

```
.report-section h4{color:#4f46e5;margin-bottom:14px;font-size:16px;border-bottom:1px solid
#e5e7eb;padding-bottom:8px}

.report-kv{display:flex;justify-content:space-between;padding:7px 0;border-bottom:1px solid
#f3f4f6;font-size:14px}

.report-kv:last-child{border:none}

.report-kv span{color:#6b7280}

.report-kv strong{color:#1f2937}



.stars span{font-size:26px;cursor:pointer;transition:transform .12s;display:inline-block}

.stars span:hover{transform:scale(1.2)}



a.lnk{color:#4f46e5;font-weight:500;text-decoration:none}

a.lnk:hover{text-decoration:underline}



#toast{position:fixed;bottom:22px;right:22px;background:#1f2937;color:#fff;padding:12px
20px;border-radius:9px;font-size:14px;z-index:9999;box-shadow:0 4px 16px rgba(0,0,0,.25);transition:opacity
.3s;opacity:0;pointer-events:none}

</style>

</head>

<body>



<div id="toast"></div>



<nav>

  <h1>Museum Management System</h1>

  <div class="nav-right" id="navRight" style="display:none">

    <span id="navUser"></span>

    <button class="btn-gray btn-sm" style="margin:0" onclick="logout()">Logout</button>

  </div>

</nav>
```

11. Configuration & Resources

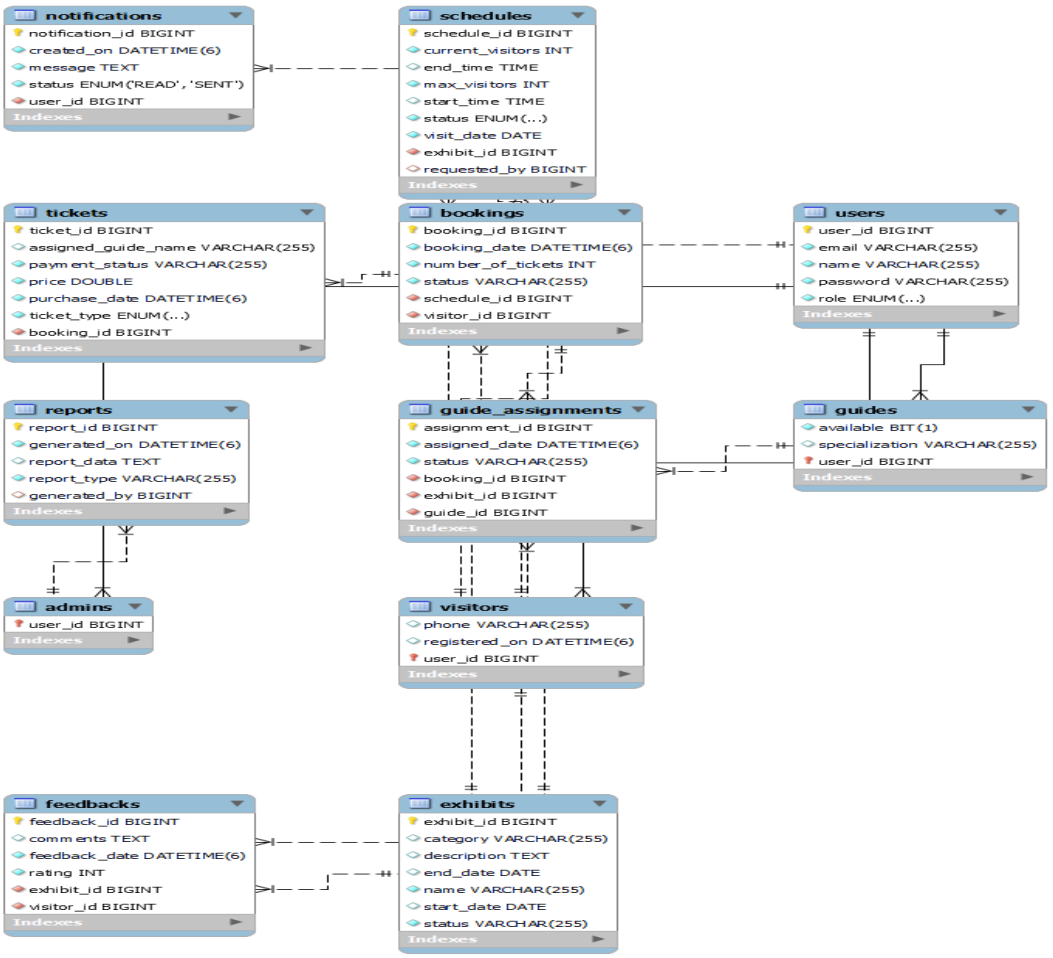
11.1 pom.xml (Maven Build)

- pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"

         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0

         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>



    <parent>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-parent</artifactId>

        <version>3.3.5</version>

        <relativePath/>

    </parent>



    <groupId>com.museum</groupId>

    <artifactId>museum-app</artifactId>

    <version>1.0.0</version>

    <name>Museum Management System</name>

    <description>Museum Management System - Spring Boot REST API</description>



    <properties>

        <java.version>17</java.version>

    </properties>



    <dependencies>

        <dependency>

            <groupId>org.springframework.boot</groupId>

            <artifactId>spring-boot-starter-web</artifactId>

        </dependency>

        <dependency>
```

```
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

DATABASE DESIGN DETAILS



DATABASE CONFIGURATION DETAILS

```
spring.datasource.url=jdbc:mysql://localhost:3306/museum_db?createDatabaseIfNot
Exist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=your_password_here
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
server.port=8080
spring.jackson.serialization.fail-on-empty-beans=false

logging.level.com.museum=INFO
```